

Generalized Star Height at Most Four

OpenAI

September 25, 2026

Abstract

Every regular language over a finite alphabet has generalized star height at most four over that same alphabet. We give a complete construction using an affine correction that hides one interval product, a finite clock, and several scales for moving boundaries through periodic words.

1 Introduction

How deeply must Kleene stars be nested to describe a regular language when complement is also available? The answer depends on the permitted operations. In this paper an expression over a fixed finite alphabet Σ starts from 0, 1, and the letters of Σ , denoting $\emptyset$, $\{\epsilon\}$, and the corresponding singleton languages. It uses union, concatenation, complement in Σ^* , and Kleene star. Its height is given by

$$\begin{aligned} \text{ht}(0) &= \text{ht}(1) = \text{ht}(a) = 0, \\ \text{ht}(P \cup Q) &= \text{ht}(PQ) = \max\{\text{ht}(P), \text{ht}(Q)\}, \\ \text{ht}(\neg P) &= \text{ht}(P), & \text{ht}(P^*) &= 1 + \text{ht}(P). \end{aligned}$$

The *generalized star height* $h_\Sigma(L)$ is the minimum height of an expression defining $L \subseteq \Sigma^*$. Finite Boolean operations preserve any common height bound. Finite languages have height zero, as does $\Sigma^* = \neg 0$. The complement operation keeps the height of its operand; it does not remove stars already nested in that operand.

Theorem 1.1. *For every finite alphabet Σ and every regular language $L \subseteq \Sigma^*$, one has $h_\Sigma(L) \leq 4$.*

We prove the theorem by a complete construction over the given alphabet. The bound is independent of Σ , L , and the number of states in a recognizing automaton. Our finite choices can be found by exhaustive search; we assert no useful bound on their sizes or on expression length.

1.1 The problem and the three constructions

The ordinary problem arose from expressions without complement. Eggan related their star height to transition graphs [2]; Dejean and Schützenberger established unbounded ordinary star height over a binary alphabet [1]. Those lower bounds do not carry over when complement is added. In the Boolean model, Schützenberger characterized the height-zero, or star-free, languages as exactly those recognized by aperiodic finite monoids [10].

Height-one results extend well beyond the aperiodic case. Henneman proved that every language recognized by a finite commutative group has height at most one, as recalled in [8, Theorem 3.4]. Pin, Straubing, and Thérien extended this conclusion to finite nilpotent groups of class two [8, Theorem 7.3]. These algebraic results do not supply a uniform bound for arbitrary finite monoids.

Absolute boundedness and the assertion that height one always suffices are different questions; Straubing explicitly formulates both [11, p. 537]. Theorem 1.1 supplies an absolute bound. It does not exhibit a language of height greater than one or decide whether one always suffices.

The alphabet convention is part of this problem. An alphabetic morphism sends each letter to a letter or the empty word. Pin, Straubing, and Thérien prove that inverse alphabetic morphisms do not increase generalized height [8, Corollary 4.7]. They also express every regular language as the inverse image, under a general morphism, of a language of ordinary height at most one [8, Theorem 8.1]. Preservation of every generalized-height bound under arbitrary inverse morphisms would therefore already imply the bound one. Our component expressions are instead constructed directly over the given alphabet; finite tags index tests and never become new input letters.

This paper and two companions give different ways to bound the nesting. *Finite Monoid Computations and a Uniform Generalized Star-Height Bound* [7, Theorem 1.1] gives height thirteen using prediction trials and a computation that records the full history of earlier tests. The present construction gives height four using an affine correction and several scales at which product boundaries may move. The companion *Generalized Star Height at Most Three* [6, Theorem 1.1] gives the stronger numerical bound three through arbitrary-function shift tables and a decoder that controls its entire set of candidate cuts. Each paper proves its own required algebra, timing statements, and original-alphabet compilation. The purpose here is to develop the complete multiscale method, including its two different residual decoders.

The split and affine-recovery framework is also developed in [7, Sections 2–4]. Our local split lemma permits both factors to have height at most one, and we prove that precise form below. The finite probability argument behind our affine correction is an application of the dependency-graph method of Erdős and Lovász [3, Section 2]. Our finite-window markers use the marker-versus-periodicity principle associated with Krieger’s marker lemma [4, Lemma 2]; the particular local rule and every periodic transport claim needed here are proved in full. These comparisons concern methods and mathematical scope: neither companion’s numerical theorem is a premise of this proof.

1.2 From products to three splits

A deterministic automaton turns each word into a transformation of a finite set. These transformations form a finite monoid M , and their product in reading order determines acceptance. We therefore seek low-height tests for the value of a morphism $T : \Sigma^* \rightarrow M$. Cancellation and commutativity are not assumed.

The word will be parsed into nonempty pieces called *episodes*, followed by a remainder with no episode prefix. Each episode has a unique end, determined by a bounded marker search or by the first departure from a periodic continuation. A graph test records the input and output of a fixed finite-state update on a sequence of episodes. To construct such a test, we split selected episodes into a prefix test and a suffix test, and use an already available graph test to skip the other episodes. The two factors choose their finite information independently. Soundness must therefore hold for every accepted pair, including pairs with different guesses and a proposed suffix that ends too soon or too late.

The first difficulty is the product of the interval containing the cut: neither side can read it in full. Suppose a segment has interval products $(d_1, \dots, d_l) \in M^l$ and carries a state in a finite vector space. We choose one correction function β on these tuples so that, for every input state and every tuple, the corrected output is constant when one coordinate d_j varies. The prefix can then transmit the input state and the earlier products; the suffix tests constancy while supplying the later products. The unknown product d_j need not be transmitted. Crucially, the same β works simultaneously for all inputs and tuples.

The second difficulty is timing. A suffix knows where it begins but may not know the origin of the episode that produced it. A finite clock restricts the possible pairs of prefix and suffix information to matching tags or one uniquely determined mismatch. A deterministic affine correction handles that mismatch. For matching tags, we choose boundaries that remain fixed near markers and can move through a periodic region without changing ordered monoid products. Several scales of movement and a polynomial-indexed set of cut positions make the number of ambiguous origin pairs smaller than the number of copies of every needed cut.

There are two remaining timing obstructions: failure of the clock or of all inner scales, and instability of the later end search. Two buffers handle them by different decoders. One first forces a common exact end and then decodes an origin; the other first decodes an origin using bounded data and then checks that origin's exact end. Their physical order lets each suffix read the complete later update table. Those tables are defined at every hypothetical monoid value, including impossible prefix values. Lifting them to linear maps on a larger basis permits recovery on whole episode sequences.

These operations lead to three applications of one split identity. Starting from the empty skipped class, their heights are $0 \rightarrow 2 \rightarrow 3 \rightarrow 4$; the intervening affine and basis recoveries use only finite Boolean operations. To justify this count we must also control every component language on its own, under false guesses as well as true ones. Each has only one unbounded periodic continuation and hence a finite cover by word templates ab^hc . Ultimately periodic tests on h compile directly over Σ . No tag or state is introduced as an extra input letter.

Organization. Section 2 gives the monoid reduction, independent-witness split principle, and affine-line lemma. Section 3 constructs the finite clock and proves its roster-independent bound. Section 4 builds the roster, scales, and moving boundaries and counts ambiguous origins. Section 5 places all windows and proves exact end detection. Section 6 defines the total updates and performs the three splits. Section 7 compiles the individual tests, handles the remainder, and completes the height calculation.

2 Algebra and split computations

This section separates the algebraic work from the later geometry. We first represent a word by its finite-monoid value. We then show how a sound split extends graph tests from a skipped class of episodes to a larger class, and how affine corrections can be removed from sequence computations. The final lemma supplies the correction that lets a split omit one interval product.

We may assume that Σ is nonempty: otherwise its only languages are $\emptyset$ and $\{\epsilon\}$, both of generalized star height zero. Fix a morphism

$$T : \Sigma^* \longrightarrow M$$

into a finite monoid, and put $m = |M|$ and $N = m!$. It suffices to construct bounded-height expressions for the fibers of T . Indeed, a deterministic finite automaton supplies such a morphism by assigning to a word its transformation of the state set, and its accepted language is a finite union of fibers.

Letters occupy unit intervals with integer endpoints. For an available interval $[a, b)$, with $a \leq b$, write $T_{a,b}$ for the product of its letters; in particular $T_{a,a} = 1_M$. Products and actions are written in reading order, so $T_{a,c} = T_{a,b}T_{b,c}$ when $a \leq b \leq c$. Let $V = \mathbf{F}_2^M$ have basis $(e_c)_{c \in M}$. Right multiplication in M induces the linear right action $e_cd = e_{cd}$, extended to all of V . The image

$e_{1_M}d = e_d$ distinguishes every $d \in M$. We will also use other finite-dimensional vector spaces over $\mathbf{F}_2$, with linear right actions written $v\rho(d)$.

One elementary consequence of finiteness will be used when boundaries move through periodic words:

$$c^{h+N} = c^h \quad (c \in M, h \geq m). \quad (2.1)$$

To see this, choose $0 \leq a < b \leq m$ with $c^a = c^b$. Multiplication by further powers gives period $b - a$ from exponent a onward, and $b - a$ divides N . This also covers $m = N = 1$.

2.1 Splitting a sequence of episodes

A *prefix code* is a set $E \subseteq \Sigma^+$ in which no word is a proper prefix of another. Every word in E^* has a unique factorization into elements of E : compare the first factors, which must coincide by the prefix condition, and continue. We call the elements of E *episodes*.

Suppose each episode e carries a deterministic update G_e of a fixed finite state set S . A *graph language* from x to y on D^* , where $D \subseteq E$, consists of those words in D^* whose successive episode updates send x to y . The empty word carries the identity update. Throughout, a graph language includes its indicated domain condition; it accepts no word outside D^* .

The next lemma receives two independently defined languages for the factors of a split. It assumes universal soundness of their concatenations but asks for a successful split only outside the skipped class. In particular, completeness does not license either side to trust the other side's guesses. The common update on an episode must be fixed before either guess is made.

Lemma 2.1 (Split principle). *Let $D' \subseteq D \subseteq E$, where E is a prefix code, and let $(G_e)_{e \in D}$ be deterministic updates of a finite set S . For $x, y \in S$, suppose languages P_x, Q_y satisfy the following conditions.*

- (i) *If a word in D^* has a prefix uv with $u \in P_x$ and $v \in Q_y$, then its first episode e exists, $uv = e$, and $G_e(x) = y$.*
- (ii) *For every $e \in D \setminus D'$ and every $x \in S$, $e \in P_x Q_{G_e(x)}$.*

Let $W_{x,y}$ be the graph languages for the same updates on $(D')^$. Then the graph language from x to y on D^* is*

$$D^* \cap \left[W_{x,y} \cup \left(\bigcup_{a \in S} W_{x,a} P_a \right) \left(\bigcup_{b,c \in S} Q_b W_{b,c} P_c \right)^* \cdot \left(\bigcup_{f \in S} Q_f W_{f,y} \right) \right]. \quad (2.2)$$

If D , all P_x , and all Q_y have height at most one, and all $W_{x,y}$ have height at most $b \geq 0$, then these new graph languages have height at most $\max\{2, b + 1\}$.

Proof. Soundness of the expression. Consider a factorization accepted by the expression on the right of (2.2). The outer intersection puts the word in D^* , so it has its true episode boundaries. A W -factor starting at such a boundary consumes a sequence of complete episodes: its factorization into members of $D' \subseteq E$ agrees with the ambient factorization by the prefix-code property.

In the second term, pair each P -factor with the following Q -factor. Condition (i) says that this pair consumes exactly the next episode and has the indicated input and output states. Induction through the alternating W -factors and these pairs proves both the boundary alignment and the

required composite update. This includes the case in which the central star contributes no factors, leaving one explicit split. The first term $W_{x,y}$ is already correct.

Completeness and height. Conversely, take a word in D^* with the required update. Split each episode outside D' using condition (ii), and absorb each intervening sequence of D' -episodes into its graph language W . This gives the second term, unless there are no explicit splits, in which case it gives $W_{x,y}$. Thus the displayed language is exact.

For the height estimate, D^* has height at most two. The central star has height at most $1 + \max\{1, b\}$, and all other operations are finite unions, concatenations, or intersections. The stated bound follows. $\square$

2.2 Recovering linear parts

The split identity has reduced the language problem to episode updates. We are free to add an episode-dependent affine correction if we can later recover the desired linear action. The next identity does that recovery on the same complete word in both tests. It is therefore valid after arbitrarily many episode updates, not merely after one.

Lemma 2.2 (Affine recovery). *Let U be a finite-dimensional vector space over $\mathbf{F}_2$ and let D be a subset of a prefix code. Suppose the update for $e \in D$ is*

$$G_e(v) = vL_e + \sigma_e,$$

where L_e is linear and $\sigma_e \in U$, both determined by the episode. If $R_{v,z}$ are the graph languages of the affine composite on D^ , then the graph language of the product of the linear parts, from v to y , is*

$$\bigcup_{f \in U} (R_{v,y+f} \cap R_{0,f}).$$

In particular this recovery does not increase a common height bound.

Proof. Applying $v \mapsto vL + c$ and then $v \mapsto vA + d$ gives $v \mapsto vLA + cA + d$. Hence the composite on any word of D^* has the form $v \mapsto vL + c$, where L is the product of the episode linear parts in reading order. Its output at zero is c . The displayed intersection says that the two outputs are $y + f$ and f , which is equivalent to $vL = y$. The union ranges over the finite set of possible f . Only finite Boolean operations are used. $\square$

2.3 An affine correction that hides one interval product

At an interior cut, neither factor can read the product across the cut. The following finite lemma provides a correction for which one interval product can be omitted. The omitted interval may depend on the input state and the full tuple, but the correction itself does not depend on the input state.

Lemma 2.3 (Affine-line constancy). *Let U be a finite-dimensional vector space over $\mathbf{F}_2$ and let ρ be a linear right action of M on U . There are an integer $l \geq 1$ and a function $\beta : M^l \rightarrow U$ such that, for every $v \in U$ and every $\mathbf{d} = (d_1, \dots, d_l) \in M^l$, some $j \in \{1, \dots, l\}$ makes the function*

$$a \longmapsto v\rho(d_1 \cdots d_{j-1} a d_{j+1} \cdots d_l) + \beta(d_1, \dots, d_{j-1}, a, d_{j+1}, \dots, d_l)$$

constant on M .

Proof. We first bound the probability of failure for one state and tuple, then show that all these failures can be avoided simultaneously. The length l is chosen only after both estimates are available.

Write $q_U = |U|$. If $m = 1$ or $q_U = 1$, take $l = 1$ and $\beta = 0$. We therefore assume $m, q_U > 1$. For an integer l to be chosen, choose the values of β independently and uniformly in U at all points of M^l . For each $(v, \mathbf{d})$, let $\mathcal{B}_{v, \mathbf{d}}$ be the event that no coordinate line through $\mathbf{d}$ has the asserted constancy.

Condition on $\beta(\mathbf{d})$. Constancy along a specified line then prescribes the values of β at its other $m - 1$ points, so has probability q_U^{1-m} . Different coordinate lines through $\mathbf{d}$ have disjoint sets of noncentral points. Their constancy events are therefore independent under this conditioning, and the conditional probability of failure does not depend on the center value. Consequently

$$\Pr(\mathcal{B}_{v, \mathbf{d}}) = p_l := (1 - q_U^{1-m})^l.$$

An event $\mathcal{B}_{v, \mathbf{d}}$ uses only the random values at points of Hamming distance at most one from $\mathbf{d}$. It is thus independent of the joint family of events whose centers have distance greater than two from $\mathbf{d}$. The number of potentially dependent events is at most

$$q_U \left(1 + l(m-1) + \binom{l}{2} (m-1)^2 \right) - 1 \leq D_l := q_U(1 + lm + l^2 m^2).$$

This count includes all possible input vectors at each nearby center.

We use a local-lemma argument [3, Section 2, pp. 616–617], recalling the required finite probability estimate with its proof. Suppose a finite family of events has a dependency graph of maximum degree at most $D \geq 1$: each event is independent of the joint family of its nonneighbors. If every event has probability at most $u(1-u)^D$, where $0 < u < 1$, then their simultaneous nonoccurrence has positive probability.

For completeness, induct on the size of a conditioning set to show that its nonoccurrence has positive probability and that the conditional probability of any other event is at most u . The assertion with an empty conditioning set follows from the probability bound. Positivity for a set of size k follows by removing one member and applying the conditional bound for size $k - 1$. To prove the conditional bound for an event A and a set S of size k not containing it, partition S into its neighbors S_1 and nonneighbors S_0 . If S_1 is empty, joint independence gives the unconditional bound. Otherwise,

$$\Pr \left(A \mid \bigcap_{B \in S} B^c \right) \leq \frac{\Pr(A \mid \bigcap_{B \in S_0} B^c)}{\Pr(\bigcap_{B \in S_1} B^c \mid \bigcap_{B \in S_0} B^c)} \leq \frac{\Pr(A)}{(1-u)^{|S_1|}} \leq u.$$

The denominator is bounded by revealing the members of S_1 successively: every conditional set then has size at most $k - 1$, so the induction applies. The numerator uses joint independence from S_0 . This completes the induction and proves the estimate.

Apply the estimate with $D = D_l$ and $u = 1/(2D_l)$. Bernoulli's inequality gives

$$u(1-u)^{D_l} = \frac{1}{2D_l} \left(1 - \frac{1}{2D_l} \right)^{D_l} \geq \frac{1}{4D_l}.$$

The quantity p_l decays exponentially in l , whereas D_l grows quadratically. Choosing l large enough makes $p_l \leq 1/(4D_l)$. There is therefore a choice of β for which no failure event occurs. Fix such a choice. $\square$

Remark 2.4 (Uniformity and finite choice). The choices of l and β precede the state v and tuple $\mathbf{d}$. Only the successful coordinate j may depend on them. The proof is also an effective finite existence argument: increase l until its displayed probability inequality holds, then enumerate the finitely many functions $M^l \rightarrow U$ and test the stated property for every state and tuple. At least one function passes. This procedure asserts no useful running-time bound.

To apply the lemma to a word segment, choose ordered checkpoints $B_0 < \dots < B_l$ and set $d_j = T_{B_{j-1}, B_j}$. Process the segment by ρ , then add $\beta(\mathbf{d})$ at B_l . If a cut lies in interval j , the prefix can transmit the vector v at B_0 and the preceding products $d_1, \dots, d_{j-1}$. The suffix supplies $d_{j+1}, \dots, d_l$ and checks constancy on the missing-coordinate line. When that check holds, it obtains the correct vector at B_l without knowing d_j . For every actual tuple and input vector at least one interval permits this procedure. Because the added vector is a function of the actual tuple alone, the update of the whole segment is affine and remains eligible for Lemma 2.2.

3 A finite tag clock

The affine-line lemma requires the two factors to communicate finite data. We call a possible collection of these data a *tag*. This section constructs a clock that controls independent tag choices by testing the two length residues, measured from scheduled reference positions. Its relation permits either matching tags or a single specified mismatch. The latter can later be absorbed into an affine correction. A second property bounds ambiguous pairs of candidate origins uniformly in their common translation. That bound must depend only on the tags, since the number of candidate origins will be chosen afterward.

Lemma 3.1 (Finite tag clock). *Let Λ be a finite nonempty set, let $\mathcal{R}$ be any finite set, and let $Q \geq 1$ be an integer. Put*

$$F_{\text{cl}} = 400, \quad C_{\text{cl}} = 2^{|\Lambda|}(1 + 4F_{\text{cl}}^2).$$

There exist a positive integer n , all of whose prime factors exceed Q , sets $I_\alpha, J_\alpha \subseteq \mathbb{Z}/n\mathbb{Z}$ for $\alpha \in \Lambda$, a set $W \subseteq \mathbb{Z}/n\mathbb{Z}$, and residues $v_p \in \mathbb{Z}/n\mathbb{Z}$ for $p \in \mathcal{R}$ with the following properties.

(i) *For every $s \in \mathbb{Z}/n\mathbb{Z}$, the relation*

$$\mathcal{E}(s) = \{(\alpha, \alpha') \in \Lambda^2 : s \in I_\alpha + J_{\alpha'}\}$$

is either a subset of the diagonal or a singleton consisting of an off-diagonal pair. If $s \notin W$, it is the full diagonal.

(ii) *For every $u \in \mathbb{Z}/n\mathbb{Z}$,*

$$\#\{(p, p') \in \mathcal{R}^2 : p \neq p', \quad u + v_{p'} - v_p \in W\} \leq C_{\text{cl}}.$$

In particular, C_{cl} depends only on $|\Lambda|$.

Proof. The construction uses one block of coordinates for every left/right assignment of the tags. First-coordinate restrictions exclude competing tag pairs, while a batch of additional coordinates excludes reverse pairs and controls translated roster differences. We prove these two tasks separately and only then identify the product group with one cyclic group.

Coordinate blocks and their sets. We first construct the sets in a finite product of cyclic groups. For each map $\chi : \Lambda \rightarrow \{\text{L}, \text{R}\}$, reserve one first coordinate and a batch of coordinates. The batch contains a dedicated coordinate for every ordered list

$$((p_a, p'_a))_{a=0}^{F_{\text{cl}}-1}$$

of ordered pairs of distinct elements of $\mathcal{R}$ such that the underlying unordered edges are distinct and form a forest. Add a spare coordinate if necessary so that the batch has positive odd size. Thus, even when no such list exists, the batch consists of one spare coordinate. There are finitely many coordinates. Give them distinct prime orders q , each satisfying

$$q > \max\{Q, 400, 2^{|\mathcal{R}|+1}\}.$$

There are enough primes: the product-plus-one argument supplies a prime outside any prescribed finite list. Write G for the resulting product of cyclic groups.

Identify a coordinate of order q with the subgroup $q^{-1}\mathbb{Z}/\mathbb{Z}$ of the circle $\mathbb{T} = \mathbb{R}/\mathbb{Z}$. Write $\|x\|_{\mathbb{T}}$ for circular distance to zero, and call a coordinate value *small* if $\|x\|_{\mathbb{T}} \leq 1/100$. Define the two sets of centers

$$C_I = \{0, 1, 3\}/6, \quad C_J = \{2, 4, 5\}/6 \quad \text{in } \mathbb{T}.$$

For each tag α , impose the following constraints in the block belonging to χ .

- If $\chi(\alpha) = \text{L}$, require the first coordinate of I_α to be zero and that of J_α to be nonzero. In every batch coordinate, require the I_α value to have distance at most $1/16$ from C_I , and the J_α value to have distance at most $1/16$ from C_J .
- If $\chi(\alpha) = \text{R}$, require the first coordinate of I_α to be nonzero and that of J_α to be zero. For each of I_α and J_α , require a strict majority of the batch coordinates to be small.

The constraints over all blocks define $I_\alpha, J_\alpha \subseteq G$. Let $W \subseteq G$ be the union, over all χ , of the following two obstructions: the first coordinate is zero; or no coordinate of the batch is small.

Sums and differences inside one batch. We verify the sum and difference properties of a batch. For a left tag, every target in each cyclic coordinate is a sum of allowed values. Indeed, $C_I + C_J$ contains all six multiples of $1/6$. Given a target t , choose $c_I \in C_I$, $c_J \in C_J$ and a signed displacement δ with $|\delta| \leq 1/12$ such that $t = c_I + c_J + \delta$ in $\mathbb{T}$. Round $c_I + \delta/2$ to a nearest grid point i , and put $j = t - i$. Both i and j are grid points, and their distances from c_I and c_J , respectively, are at most

$$\frac{1}{24} + \frac{1}{2q} < \frac{1}{24} + \frac{1}{800} < \frac{1}{16}.$$

Moreover, the distance between any center in C_I and any center in C_J is at least $1/6$. Consequently every allowed left-tag difference $i - j$ satisfies, in every batch coordinate,

$$\|i - j\|_{\mathbb{T}} \geq \frac{1}{6} - \frac{2}{16} = \frac{1}{24}.$$

For a right tag, any target batch with one small coordinate is a sum of allowed batches. To see this, write the batch size as $2k + 1$ and choose a small target coordinate. At that coordinate, assign the target to the first summand and zero to the second. Partition the other $2k$ coordinates into two sets of size k . On the first set put zero in the first summand and the target in the second; on the second set do the reverse. Both summands then have at least $k + 1$ small coordinates. Conversely, any two allowed right-tag batches have a common small coordinate, because their sets of small coordinates are strict majorities. Their difference therefore has some coordinate satisfying

$$\|i - j\|_{\mathbb{T}} \leq \frac{2}{100} < \frac{1}{24}.$$

Thus the difference sets $I_\alpha - J_\alpha$ and $I_{\alpha'} - J_{\alpha'}$ are disjoint whenever a block places α on the left and α' on the right.

The full relation, including independently mismatched tags. If $s \notin W$, all first coordinates of s are nonzero and every batch has a small coordinate. The preceding sum constructions, together with the first-coordinate decompositions $s = 0 + s$ and $s = s + 0$, show that $s \in I_\alpha + J_\alpha$ for every tag α . The choices in distinct blocks are independent, so they combine to give elements of G .

Now suppose an off-diagonal pair (α, α') belongs to $\mathcal{E}(s)$. Consider any competing pair (λ, μ) with $\{\lambda, \mu\} \neq \{\alpha, \alpha'\}$. There is an assignment χ placing α on the left, α' on the right, and λ, μ on the same side: identifying λ with μ does not identify α with α' . In its first coordinate the pair (α, α') forces $s = 0$, whereas the pair (λ, μ) requires $s \neq 0$. Hence that competitor is impossible. The only other possible competitor is the reverse pair (α', α) . If both pairs represented s , there would be

$$i_\alpha + j_{\alpha'} = i_{\alpha'} + j_\alpha, \quad \text{hence} \quad i_\alpha - j_\alpha = i_{\alpha'} - j_{\alpha'},$$

with the indicated elements belonging to their tag sets. A block separating these tags contradicts the disjointness of the difference sets. This proves the first alternative in (i). It also excludes off-diagonal pairs outside W , where the full diagonal is present, and completes the proof of (i).

Assigning the roster and counting whole-batch obstructions. The tag relation is now established. What remains is a choice of the v_p that makes membership in W rare among ordered roster differences, for every common translate. Write $r = |\mathcal{R}|$ and enumerate the roster as $p_0, \dots, p_{r-1}$. In every first coordinate set the residue of v_{p_j} equal to 2^j . All ordered differences for distinct roster slots are distinct modulo that coordinate's prime. For $r \geq 2$, they are distinct as integers: the sign determines their orientation, and in a positive difference

$$2^b - 2^a = 2^a(2^{b-a} - 1), \quad a < b,$$

the power of two dividing it determines a , after which the odd factor determines b . Reduction modulo q creates no collision, since the distance between any two such integer differences is less than $2^r < q$. The assertion is vacuous when $r < 2$. For any translate u , at most one ordered pair therefore hits a specified first-coordinate zero obstruction.

In the dedicated batch coordinate for a list $((p_a, p'_a))_{a=0}^{F_{\text{cl}}-1}$, let z_a be a grid point at circular distance at most $1/(2q)$ from a/F_{cl} . Prescribe

$$(v_{p'_a} - v_{p_a})|_{\text{coordinate}} = z_a \quad (0 \leq a < F_{\text{cl}}).$$

These prescriptions can be satisfied: choose a root value in each tree and assign successive vertex values along its edges, with the sign determined by the prescribed orientation. A forest has no consistency condition around a cycle. Assign arbitrary values to any remaining vertices and in spare coordinates. All these assignments are made independently in each coordinate, and so define elements $v_p \in G$.

For any coordinate translate t , the equally spaced points a/F_{cl} include one at distance at most $1/(2F_{\text{cl}})$ from $-t$. The corresponding prescribed difference satisfies

$$\|t + z_a\|_{\mathbb{T}} \leq \frac{1}{2F_{\text{cl}}} + \frac{1}{2q} < \frac{1}{400} < \frac{1}{100}.$$

Thus every dedicated list has a pair whose translated difference is small in its dedicated coordinate, uniformly in the translate.

Fix $u \in G$ and a batch. Call an ordered pair (p, p') , $p \neq p'$, bad for this batch if $u + v_{p'} - v_p$ has no small coordinate in the batch. Suppose there were more than $4F_{\text{cl}}^2$ such pairs. Form the simple undirected graph consisting of their underlying edges, discarding isolated vertices. If a spanning forest had fewer than F_{cl} edges, its nontrivial components would together have at most $2F_{\text{cl}} - 2$

vertices: a component with $h \geq 2$ vertices contributes $h - 1$ forest edges and satisfies $h \leq 2(h - 1)$. The number of ordered pairs would then be at most

$$(2F_{\text{cl}} - 2)(2F_{\text{cl}} - 3) < 4F_{\text{cl}}^2,$$

a contradiction. Select F_{cl} edges of a spanning forest and orient each as one of the available bad pairs. In any order, they form a list with a dedicated coordinate in this batch. The preceding covering estimate makes one of these pairs small in that coordinate, again a contradiction. Each batch therefore has at most $4F_{\text{cl}}^2$ bad ordered pairs. This conclusion also applies when the roster is too small to admit a dedicated list, by the same counting argument.

There are $2^{|\Lambda|}$ blocks. Taking the union of their first-coordinate and batch obstructions gives, for every $u \in G$,

$$\#\{(p, p') : p \neq p', u + v_{p'} - v_p \in W\} \leq 2^{|\Lambda|}(1 + 4F_{\text{cl}}^2).$$

For the empty roster the left side is zero throughout the construction.

One cyclic clock. Finally, let n be the product of all coordinate primes. The residue map from $\mathbb{Z}/n\mathbb{Z}$ to G is an injective homomorphism, since the distinct primes are pairwise coprime, and both groups have n elements. It is therefore an isomorphism. Transport all constructed sets and residues through it. This preserves sums, differences, and the two properties, while every prime factor of n exceeds Q . All sets and choices in the construction are finite. $\square$

4 Markers and several scales of cuts

The main split will use a bounded family of possible cuts. The clock controls which tags can meet at a cut, but a suffix still has several possible origins for its computation. We therefore choose boundaries whose interval products remain unchanged under suitable movements of the origin. Markers keep a boundary fixed outside periodic regions; inside such a region we may move it by a multiple of a sufficiently large period. Several scales of movement ensure a uniform bound on the number of ambiguous origins.

There are three distinct outputs. The marker rule recognizes regions where letters determine a common periodic extension. The roster and scale construction provides many cut positions of every required kind. Finally, product transport and a descending count show that only a bounded number of ordered pairs of those positions can give obstructed origins. We develop the finite data in that order and record exactly which later choices may still be enlarged.

All parameters in this section are finite and may depend on M and Σ ; lengths, positions, and thresholds are integers. We first work on two-sided words, indexed by the integers. On a finite argument, every window prescribed by a rule must be present. An unavailable window causes rejection; in particular, it never removes an origin from a universal or uniqueness test. Section 5 will place all the required windows inside complete episodes.

4.1 Local markers and periodic extensions

A positive integer a is a *period* of a finite word if its letters at distance a agree whenever both positions belong to the word.

The marker-versus-local-periodicity viewpoint goes back to Krieger's marker lemma [4, Lemma 2]; see also the finite clopen merging formulation in Meyerovitch [5, Section 3, Lemmas 3.2–3.4]. We prove the finite-window rule needed here. The long-block hypothesis $K > 3d^2$ also prepares the overlap argument in the next lemma.

Lemma 4.1 (Local markers). *Let $d \geq 2$ and $K > 3d^2$ be integers. There is a translation-equivariant rule marking integer positions of any two-sided word such that distinct markers are at distance at least d . The decision at z uses only $[z - r, z + r]$, for a fixed finite integer $r \geq K$. If there is no marker in the inclusive integer interval $[z - d, z + d]$, then the block $[z, z + K)$ has a positive period less than d .*

Proof. List the finitely many length- K words having no positive period below d . Process these words in a fixed order. When processing a word, mark all its starts that are not within distance less than d of a previously installed marker. Two starts added at this same step cannot be at distance $a < d$: their overlap would make a a period of the listed word. Induction therefore proves separation of all markers.

Each step makes its decision from a length- K block and the decisions of earlier steps at offsets of magnitude less than d . Induction over the finite list gives a finite inspection radius; enlarge it to at least K . The construction commutes with translations. Finally, a listed word at z was either marked or prevented by a marker within distance less than d . Thus marker absence on $[z - d, z + d]$ excludes every listed word at z , as required. $\square$

Lemma 4.2 (Common periodic extensions). *A block of length $K > 3d^2$ having a positive period below d determines a unique two-sided periodic extension of period below d . Two such blocks whose starts differ by less than $K - d^2$ determine the same extension. For the marking rule of Lemma 4.1, if there is no marker in $[z - H, z + H]$ and $H \geq d$, the actual letters on*

$$[z - H + d, z + H - d + K)$$

belong to one such periodic extension.

Proof. A block of period a extends by repeating its first a letters. Two two-sided words of respective periods a, b that agree on ab consecutive positions agree everywhere: both have period ab . Here $a, b < d$, so $ab < d^2$. This proves both uniqueness and the assertion about overlapping blocks.

For the last assertion, every start in the inclusive interval $[z - H + d, z + H - d]$ has no marker within distance d . Its length- K block therefore has a period below d . Consecutive such blocks overlap sufficiently, so their extensions agree. Their union is the displayed interval, on which the extension agrees with the actual letters. $\square$

Whenever these lemmas are applied to a finite word, extend the word arbitrarily in both directions. If all prescribed inspection windows are internal, the computed markers and periodic blocks are independent of that extension.

4.2 Algebraic data and the order of the buffers

Before choosing cut positions, we fix the states and finite information that each cut must transmit. Their cardinalities determine how many candidate cut positions we need.

Put

$$U_0 = V, \quad U_{i+1} = \mathbf{F}_2^{U_i \times M} \quad (i = 0, 1).$$

We denote the basis vector of U_{i+1} indexed by (v, c) by $[v, c]$. Its work action is

$$[v, c]\rho_i(d) = [v, cd] \quad (d \in M). \quad (4.1)$$

Thus the formal label $[v, c]$ retains an input state v while its second coordinate accumulates a prefix product. Once the later part of an episode is known, a terminal linear map will send each label to a

basis label recording the completed update corresponding to c . Lemma 6.2 constructs that map for every $c \in M$, including values not realized by an actual prefix. The two larger spaces will let us perform this encoding twice.

Choose l, β from Lemma 2.3 for the action of M on V . For each $i = 0, 1$, choose l_i, β_i from the same lemma for the action ρ_i on U_{i+1} . These choices are made once and for all. Thus $\beta : M^l \rightarrow V$ and $\beta_i : M^{l_i} \rightarrow U_{i+1}$.

An episode beginning at s will have a later origin

$$t = s + L.$$

Before t we will place two buffers in the order 1, 0. Buffer i has checkpoints

$$B_{i,0} < B_{i,1} < \dots < B_{i,l_i} < a_i,$$

where a_i is after all its activities. The entire buffer 1, including a_1 , precedes buffer 0. The checkpoints are translates of fixed offsets from s ; we use $B_{i,j}$ and a_i for their absolute positions in the word. The translations and L will be fixed in Section 5.

The *kind* of a cut in buffer i is a tuple

$$(j, v, d_1, \dots, d_{j-1}), \quad 1 \leq j \leq l_i, \quad v \in U_{i+1}, \quad d_f \in M. \quad (4.2)$$

Let κ_i be the number of these kinds. A finite stencil $\mathcal{P}_i$ will specify the cut offsets p from s , with each offset assigned a kind. An offset of interval number j must satisfy $B_{i,j-1} < s + p < B_{i,j}$. Write

$$h_i = \text{diam } \mathcal{P}_i, \quad \Delta_i = \mathcal{P}_i - \mathcal{P}_i. \quad (4.3)$$

At an actual cut $s + p$, a suffix considering an offset p' will infer the origin $t + p - p'$. We construct $\mathcal{P}_0$ here and $\mathcal{P}_1$ in Section 5.

Once separated numerical offsets have been chosen, kinds can be assigned in increasing interval-number order. Boundaries between successive interval numbers are then placed in the gaps between their offsets; the extreme boundaries and a_i can be added outside them. Thus it suffices to produce separated offsets with sufficiently many copies of each kind.

4.3 The clock tags and a polynomial roster

The first stencil must contain more copies of every kind than the total number of wrong ordered origin pairs that can survive our tests. The following polynomial roster leaves this growth margin: its number of slots grows by a higher power of the field size than the nonclock obstruction budget does. The exponents and other constants are fixed before that field size is selected.

For the main segment set

$$Z = l + 1, \quad k_0 = 2Z + 1, \quad R = 2k_0 - 1. \quad (4.4)$$

There will be Z centers for its boundaries and R scales for each cut role. Use the following tag set in Lemma 3.1:

$$\alpha = (j, r, v, d_1, \dots, d_{j-1}, X, Y), \quad \begin{array}{l} 1 \leq j \leq l, \quad 1 \leq r \leq R, \\ v, X, Y \in V, \quad d_f \in M. \end{array} \quad (4.5)$$

Its ambiguity constant C_{cl} is determined by the tag set alone, before a roster or its positions have been chosen.

Choose a prime $P > R$ large enough that

$$\left\lfloor \frac{P^{k_0}}{\kappa_0} \right\rfloor > C_0, \quad C_0 = C_{\text{cl}} + 2^R(3Z + 1)^R P^{2Z}. \quad (4.6)$$

Such primes exist because $k_0 = 2Z + 1 > 2Z$. The abstract roster consists of all polynomials of degree less than k_0 over $\mathbf{F}_P$, including the zero polynomial. This is the polynomial-evaluation construction underlying Reed–Solomon codes [9]; we use only the elementary root bound, which applies over the prime field chosen here. Choose distinct $\xi_1, \dots, \xi_R \in \mathbf{F}_P$ and give a roster slot p the digits

$$x_r(p) = p(\xi_r).$$

We identify field elements with $\{0, \dots, P-1\}$ when using them as digit indices. A nonzero polynomial of degree less than k_0 has at most $k_0 - 1$ roots, by successive division by linear factors. Consequently any k_0 evaluations determine a slot, and two distinct slots differ in at least

$$R - k_0 + 1 = k_0 \quad (4.7)$$

digits.

Apply Lemma 3.1 to this roster, avoiding all primes at most N . Fix its modulus n , tag sets I_α, J_α , bad set W , and roster residues $v_p \in \mathbb{Z}/n\mathbb{Z}$. The prime divisors of n are all greater than N . Notice that the modulus is chosen after the roster size; enlarging it does not change C_0 .

4.4 Scale choices and the first stencil

The abstract roster is now fixed, as are its required clock residues. We next realize the slots at integer positions. Large separation at each scale will dominate all smaller-digit changes, while a small residue correction preserves the clock assignment.

Call a positive integer *smooth* if all its prime divisors divide N . We choose thresholds $S_r < S'_r$, a positive step δ_r , a movement bound $R_r = n\delta_r$, and digit positions $f_r(0) < \dots < f_r(P-1)$, in increasing order of r . For each r perform the following choices:

- (i) Choose $S_r \geq 1$, also requiring $S_r \geq S'_{r-1}$ if $r > 1$. Choose δ_r smooth and divisible by $N\sigma$ for every smooth positive integer $\sigma \leq S_r$. Then

$$\gcd(\delta_r, n) = 1. \quad (4.8)$$

- (ii) If h'_b denotes the diameter of the digit positions at scale b , put

$$J_r = n + \sum_{b < r} h'_b.$$

Choose the digit positions so that all ordered differences of unequal digits are distinct and separated from one another and from zero by more than $10(J_r + R_r + 1)$. Denote their diameter by h'_r .

- (iii) Choose

$$S'_r > \max\{S_r, 10(h'_r + J_r + R_r + 1)\}. \quad (4.9)$$

The step in (i) may be the least common multiple of the finitely many required $N\sigma$; it is smooth. If $N = 1$, the only smooth positive integer is 1, and $\delta_r = 1$ is permissible. The digit positions in (ii) can be sufficiently scaled powers of two. Indeed a positive difference $2^a - 2^b$, $a > b$, determines b by

its power-of-two valuation and then determines a . All nonzero ordered differences are thus distinct, and scaling provides the required separation.

Assign slot p the numerical offset

$$\sum_{r=1}^R f_r(x_r(p)) + \epsilon_p, \quad 0 \leq \epsilon_p < n. \quad (4.10)$$

Choose ϵ_p to give residue v_p modulo n . For two slots whose highest differing digit is at scale r , the sum of all smaller-coordinate differences and the correction difference has absolute value less than J_r . The difference at scale r has absolute value greater than $10(J_r + R_r + 1)$. The resulting positions are therefore distinct, and every two are separated by more than 2.

These positions, up to a common translation, form $\mathcal{P}_0$. We henceforth identify a slot with its numerical offset when taking differences. Their difference residues are exactly the prescribed $v_p - v_{p'}$, including after the common translation. By (4.6), we can assign more than C_0 offsets to each stage-0 kind. Assign them in interval-number order and insert the buffer boundaries as described above. Translation of the complete configuration later changes none of its difference properties. In particular,

$$h_0 < n + \sum_r h'_r. \quad (4.11)$$

4.5 Inner controls and the main boundaries

We have fixed the stencil and every movement range. We can now make the inner markers sparse relative to all of them. In marker-free regions the boundary rule must preserve a congruence class modulo a full monoid period; choosing only a phase modulo the word period would not suffice.

Choose integers

$$d_o > 10 \left(1 + n + \sum_r h'_r + \sum_r R_r \right), \quad K_o > 3d_o^2, \quad (4.12)$$

and fix the marking rule of Lemma 4.1 with these parameters and radius $r_o \geq K_o$. Choose an integer H with

$$H > 10((N + m + 1)d_o + \max_r R_r + 1). \quad (4.13)$$

For an inner center z , let $u(z)$ be the first marker in $[z - H, z + H]$, or $\perp$ if none exists. If $u(z) \neq \perp$, put $B(z) = u(z)$.

If $u(z) = \perp$, Lemma 4.2 supplies a periodic extension of the block starting at z . Let $d < d_o$ be its least period, and let σ be the largest smooth divisor of d . Among the length- d period words starting at its different phases, choose the lexicographically least, using a fixed order on Σ . Its starts form one class $x \bmod d$: equality of two rotations would contradict minimality of d . Define $y(z)$ to be the largest integer at most z in $x + \sigma\mathbb{Z}$, and let $B(z)$ be the least integer at least z satisfying

$$B(z) \equiv x \pmod{d}, \quad B(z) \equiv y(z) \pmod{\sigma N}. \quad (4.14)$$

The definition is independent of the representative of x . Moreover,

$$\gcd(d, \sigma N) = \sigma, \quad \text{lcm}(d, \sigma N) = dN,$$

because d/σ is coprime to N . The two residues in (4.14) agree modulo σ , so they define one class modulo dN . In particular,

$$z \leq B(z) < z + dN. \quad (4.15)$$

All these choices commute with translations of the letters. They do not use an external choice of position origin. In both the present and absent cases, $B(z) \in [z - H, z + H]$.

Choose an even integer

$$G > 10(H + r_o + \max_r R_r + 1). \quad (4.16)$$

For the main segment set

$$o_j = (j + 1)G, \quad z_j = t + o_j, \quad B_j = B(z_j) \quad (0 \leq j \leq l), \quad c_j = \frac{o_{j-1} + o_j}{2} \quad (1 \leq j \leq l).$$

At another proposed origin τ , write $z_j(\tau) = \tau + o_j$ and $B_j(\tau) = B(z_j(\tau))$; the notation without an argument refers to the true origin t . The main cut offsets from t form the finite set

$$\mathcal{H} = \{c_j + \delta_r a : 1 \leq j \leq l, 1 \leq r \leq R, 0 \leq a < n\}. \quad (4.17)$$

A cut of role (j, r) lies strictly between B_{j-1} and B_j . All inspection windows for boundaries through $j - 1$ are before this cut. Conversely, for a suffix beginning at k and any hypothesized role (j, r) , a candidate origin $k - c_j - \delta_r a$ has its first required center at

$$k + G/2 - \delta_r a. \quad (4.18)$$

Since $\delta_r a < R_r$, (4.16) puts the inspections for this center and all later centers after k . This holds even when the suffix role differs from the prefix role. Complete placement of these controls, including the later end controls, is deferred to Lemma 5.2.

4.6 Usable scales and transport of products

The boundary rule is defined at every origin. We next specify when a whole range of shifted origins gives equal interval products. The condition compares actual marker positions, including absence, and in periodic regions controls the phase lattice used by the congruence rule.

Definition 4.3. A scale r is *usable at the origin t* if, at each of its Z centers z_j , the following conditions hold:

- (i) The value $u(z_j + h)$ equals $u(z_j)$ for every integer h with $|h| \leq R_r$, including agreement on absence.
- (ii) If $u(z_j) = \perp$, then either $\sigma \leq S_r$, or the inclusive interval $[z_j - R_r, z_j + R_r]$ contains no point of $x + \sigma\mathbb{Z}$.

Here d, σ, x refer to the periodic extension at the indicated center. All windows needed by these checks are mandatory.

Lemma 4.4 (Transport of boundary products). *Suppose r is usable at t , and h is a multiple of δ_r with $|h| \leq R_r$. Write $B_j^{(h)} = B(z_j + h)$. Then*

$$T_{B_{j-1}, B_j} = T_{B_{j-1}^{(h)}, B_j^{(h)}} \quad (1 \leq j \leq l). \quad (4.19)$$

For a fixed endpoint $b \geq z_l + H$, the last transfer also satisfies

$$T_{B_l, b} = T_{B_l^{(h)}, b}. \quad (4.20)$$

These statements apply to finite words whenever the prescribed windows and product intervals are available. They hold for arbitrary finite monoids, without a commutativity assumption.

Proof. The proof has three steps: the two boundaries stay in one class modulo dN ; the actual word contains enough periodic letters on both sides; and monoid-power periodicity then compares the ordered products.

A common boundary class. Consider one center z . If its marker is present, usability leaves the boundary unchanged. If absent, the old and new periodic blocks have the same extension by Lemma 4.2: $|h| \leq R_r < d_o$, and the blocks have length $K_o > 3d_o^2$. Thus the least period d , smooth divisor σ , and phase class $x \bmod d$ are common.

If $\sigma \leq S_r$, the displacement h is divisible by σN . Translation along the lattice gives $y(z+h) = y(z) + h$, so the second residue in (4.14) is unchanged. Otherwise, usability says that no point of that lattice lies between any of the positions under consideration, so $y(z+h) = y(z)$. In both cases the two boundaries belong to one class modulo dN . Writing them in increasing order as $b_- \leq b_+$, we have

$$b_+ - b_- = kdN \quad (k \geq 0). \quad (4.21)$$

Margins of actual periodic letters. By (4.15), both choices lie in $[z - R_r, z + R_r + Nd)$. Marker absence and Lemma 4.2 give a common run of actual letters containing $[z - H + d_o, z + H - d_o)$. The inequality (4.13) therefore implies that

$$[b_- - md, b_+ + md) \subset [z - H + d_o, z + H - d_o) \quad (4.22)$$

lies in that run. In particular both choices have the required periodic margins on their two sides.

Comparing products one endpoint at a time. Let $c \in M$ be the product of one length- d period beginning in the phase of b_- . For an incoming product from any fixed $a \leq z - H$, comparison at $b_- - md$ gives the two factorizations

$$T_{a,b_-} = Ac^m, \quad T_{a,b_+} = Ac^{m+kN}.$$

For an outgoing product to any fixed $b \geq z + H$, comparison at $b_+ + md$ gives

$$T_{b_-,b} = c^{m+kN}C, \quad T_{b_+,b} = c^mC.$$

Here A and C are the products on the unchanged remaining portions. Equation (2.1) gives $c^{m+kN} = c^m$. The equalities require neither cancellation nor reordering of factors.

The old and new boundary at each center lie within its unshifted window $[z_j - H, z_j + H]$, by the same estimates. Adjacent center windows are disjoint and separated by more than their required margins, since $G > 10H$. To compare an interval between successive boundaries, move its left endpoint while fixing its right endpoint, then move the right endpoint while fixing the new left endpoint. Each fixed opposite endpoint is outside the relevant window, so the local calculation applies. This proves (4.19). Its outgoing version at the last center proves (4.20). $\square$

4.7 A uniform ambiguity bound for the first stencil

Product transport handles an origin whenever at least one scale is usable. We now bound all the pairs of stencil slots that can produce an origin where this fails, also allowing a bad clock residue. Throughout this count the anchor q is fixed. Permitting it to vary with a candidate would not give the translated-difference problem solved by the clock.

For a proposed origin t and an anchor position q , define

$$A_0(t, q) : \quad q - t \pmod{n} \in W \quad \text{or no scale is usable at } t. \quad (4.23)$$

The anchor is held fixed when comparing different proposed origins.

Proposition 4.5 (Inner ambiguity). *On any two-sided word and for any integers t, q ,*

$$\#\{(p, p') \in \mathcal{P}_0^2 : p \neq p', A_0(t + p - p', q)\} \leq C_0. \quad (4.24)$$

The same bound holds on finite words whenever all the indicated inspection windows are internal.

Proof. We count numerical difference tuples rather than choose digit pairs at every stage. Nonzero differences recover the slots; zero differences contribute only one value. This distinction prevents an unnecessary factor of P at each equal-digit coordinate.

Clock failures and recovery of the ordered pair. The clock part contributes at most C_{cl} , because

$$q - (t + p - p') \equiv (q - t) + v_{p'} - v_p \pmod{n}.$$

This is precisely the uniform translated-difference bound of Lemma 3.1. It remains to count pairs for which no scale is usable.

For an ordered pair of slots define its numerical digit differences

$$D_r = f_r(x_r(p)) - f_r(x_r(p')).$$

The full tuple $(D_1, \dots, D_R)$ determines a distinct ordered pair $p \neq p'$. Indeed, it has at least k_0 nonzero coordinates by (4.7). Each such coordinate determines both ordered digits by the uniqueness of the digit differences. These k_0 evaluations determine each polynomial slot separately; unrevealed equal digits at zero coordinates therefore introduce no additional pairs.

Exceptional scales. For a pair with no usable scale, look at its origin $t + p - p'$ and declare a coordinate r exceptional if

$$S_r < \sigma \leq S'_r \quad (4.25)$$

at at least one absent center. At this fixed origin each center has one value of σ , and the intervals $(S_r, S'_r]$ are disjoint. There are therefore at most Z exceptional coordinates. We may sum over all their possible sets, of which there are at most 2^R .

Descending count with all smaller digits still free. Fix such a set of exceptions. Count difference tuples in descending coordinate order. At an exceptional coordinate there are at most P^2 possible differences. At a nonexceptional coordinate r , fix the differences at all larger coordinates. For each center label j , its possible positions are of the form

$$C_j + D_r + e, \quad |e| \leq J_r, \quad C_j = t + o_j + \sum_{b>r} D_b. \quad (4.26)$$

The error includes the smaller-coordinate differences and $\epsilon_p - \epsilon_{p'}$. This description fixes only the larger numerical differences; it does not assume that their underlying digit pairs have already been determined.

Since scale r fails to be usable, at some center label one of the following occurs:

- (a) A marker lies within R_r of one of the two search endpoints.
- (b) The search is absent at the center, $\sigma > S'_r$, and a point of $x + \sigma\mathbb{Z}$ lies within R_r of the center.

Indeed a change in the first marker of a translated search interval requires a marker to enter or leave through an endpoint. If the search does not change, failure of the second usability condition requires absence, $\sigma > S_r$, and such a lattice point. Nonexceptionality then strengthens $\sigma > S_r$ to $\sigma > S'_r$.

For (a), the range swept by a given search endpoint, enlarged by R_r , has diameter at most

$$2(h'_r + J_r + R_r) < d_o. \quad (4.27)$$

It contains at most one marker. A fixed marker can serve at most one nonzero value of D_r : by (4.26), serving it restricts D_r to an interval of length $2(J_r + R_r)$, whereas distinct nonzero digit differences are more widely separated. The two endpoints give at most two such values for each label.

For (b), all the centers under consideration for a fixed label lie in an interval of diameter at most $2(h'_r + J_r) < d_o$. Among those centers whose searches are absent, their length- K_o blocks determine the same two-sided periodic extension, by Lemma 4.2: their start distance is less than d_o , and $K_o - d_o > d_o^2$. Only absent candidates enter this comparison. Present candidates between them are irrelevant, since overlap directly compares any two of the absent candidates' seed blocks. Thus, even as D_r , the lower digits, and the residue corrections vary, these absent candidates have one common least period, smooth divisor, phase, and lattice. This is where the long local blocks are needed in the descending count. If the common σ does not exceed S'_r , none of these candidates can realize (b). Otherwise the enlarged center range has diameter less than σ , by (4.9); it contains at most one point of the common lattice. That point accounts for at most one nonzero difference D_r , by the same separation argument as for a marker.

Summing the budgets. There are Z center labels. Allowing also the single difference value zero, a nonexceptional coordinate therefore has at most $3Z + 1$ possibilities. It is not necessary to specify the equal digits when the difference is zero. Summing the descending count over exception sets bounds the number of full difference tuples by

$$2^R P^{2Z} (3Z + 1)^R.$$

Since different ordered pairs give different tuples, this also bounds the number of pairs. Adding the clock contribution proves (4.24) with the constant in (4.6).

For a finite word, take any two-sided extension. Internal inspection windows give the same decisions, so the bound just proved applies unchanged. $\square$

The order of all choices is worth recording. The affine data and tag set fix C_{cl} ; then P fixes the roster; then the clock fixes n ; then the scale parameters and digit positions are chosen successively. Only after these choices do we choose the inner marker parameters, H , and G . Thus neither the ambiguity estimate nor the number of copies per kind depends circularly on a later geometric choice. The remaining buffer, end controls, and translations can now be added around these fixed finite arrangements.

5 Episodes and exact end detection

The first ambiguity estimate held its anchor fixed. We must now arrange that independently accepted suffix tests actually use that same anchor. This section builds an episode rule and proves the necessary exact-end statement. The main split and the first residual split compare a full list of bounded marker searches, then run one unbounded end search at a reference origin. The second residual split uses a different order: bounded tests decode an origin before its one end search. Keeping these orders distinct is necessary both for soundness and for the later height-one compilation.

We first construct the second stencil and bound instability of its end search. We then place every window, including windows at wrong candidate origins, inside a complete episode. The final guard lemma treats arbitrary accepted suffixes, not merely the intended suffix of that episode.

5.1 The second stencil and the end obstruction

Retain the stage-0 stencil and main cut set from Section 4. Choose an integer

$$w > \max\{h_0, \text{diam } \mathcal{H}\}, \quad C_1 = 2|\Delta_0|.$$

For each stage-1 kind, take more than C_1 copies. Give the copies distinct integer offsets whose nonzero ordered differences are all distinct and are separated from one another and from zero by more than $2w + 10$. Scaled powers of two provide such offsets: the positive difference $2^b - 2^a$, $b > a$, determines a from its exact power of two and then determines b ; a sufficiently large common scale gives the separation. Assign the kinds to the offsets in interval-number order and insert the buffer boundaries in the intervening gaps, as in Section 4. Denote the resulting stencil by $\mathcal{P}_1$, and put

$$h_1 = \text{diam } \mathcal{P}_1, \quad \Delta_1 = \mathcal{P}_1 - \mathcal{P}_1.$$

We will translate each entire buffer arrangement later; translations do not affect these difference properties.

Choose integers d_e, K_e with

$$d_e > 10(1 + w + h_1), \quad K_e > 3d_e^2,$$

and apply Lemma 4.1 to obtain an end-marker rule of radius $r_e \geq K_e$. An end-search center will have the form $z_e(t) = t + o_e$, where o_e is chosen below. Let $\eta(t)$ be the first end marker in the inclusive interval $[z_e(t) - d_e, z_e(t) + d_e]$, or $\perp$ if none exists. Define the bounded predicate

$$A_1(t) \iff \eta(t + u) \neq \eta(t) \text{ for some integer } |u| \leq w. \quad (5.1)$$

Present markers are compared as positions in the word, not as offsets from their respective search centers.

Lemma 5.1 (End ambiguity). *On a two-sided word, uniformly in t ,*

$$\#\{(p, p') \in \mathcal{P}_1^2 : p \neq p', \exists \gamma \in \Delta_0 \ A_1(t + p - p' + \gamma)\} \leq C_1. \quad (5.2)$$

The same estimate holds on a finite word whenever all the indicated inspection windows are present.

Proof. If the first marker in a moving search interval changes under a shift of magnitude at most w , a marker must cross one of its endpoints. In particular, a marker lies within distance w of an original endpoint. Fix $\gamma \in \Delta_0$ and fix one of the two endpoints. As $p - p'$ ranges over Δ_1 , the possible marker positions just described lie in an interval of diameter at most $2h_1 + 2w < d_e$. This interval contains at most one end marker. For that marker, the required ordered difference $p - p'$ lies in an interval of length $2w$. The separation of the nonzero ordered differences gives at most one ordered pair with $p \neq p'$. There are two endpoints and $|\Delta_0|$ choices of γ , proving the bound. With all windows available in a finite word, take any two-sided extension; none of the inspected marker decisions changes. $\square$

5.2 Placement and the episode rule

The end rule must allow arbitrarily long episodes while exposing just one unbounded continuation. It does so by following a short periodic seed to its first mismatch. A fixed tail after the anchor leaves room for every bounded control window, even if that mismatch occurs immediately.

At a start s with origin $t = s + L$, the prospective anchor is defined as follows. If $\eta(t)$ is present, set $q = \eta(t)$. Otherwise the seed

$$[z_e(t), z_e(t) + K_e)$$

has a period less than d_e by Lemma 4.1, and a unique two-sided periodic extension by Lemma 4.2. Set q equal to the first letter position after the seed at which the word differs from this extension, if such a position exists. For a fixed positive integer b_{tail} , the prospective episode end is $q + b_{\text{tail}}$. An absent marker and no mismatch yet do not define a complete episode.

We use the following convention in every finite-argument test in the paper. A letter inspection requires its entire window to lie in the argument; a product requires its two endpoints to be available and ordered. A universal or uniqueness test on a finite candidate list requires the bounded control windows for *every* candidate. If any such window is missing, the test rejects; its candidate list is never shortened. A later computation at a decoded candidate also requires its own data. These requirements are finite checks, and are part of the languages being defined.

Lemma 5.2 (Placement). *The translations of the two buffers and the integers L, o_e, b_{tail} can be chosen so that the following properties hold.*

- (i) *After s , the buffers occur in the order $1, 0$, followed by all relevant inner-system windows and cuts, and then by all relevant end-system windows and seeds.*
- (ii) *All bounded windows required at main hypotheses $k - h$, $h \in \mathcal{H}$, or at aligned hypotheses $k + L - p'$, $p' \in \mathcal{P}_i$, are internal to every complete episode at a true cut. This includes all usability checks, the additional $\gamma \in \Delta_0$ translates at stage 1, and all shifts in A_1 .*
- (iii) *A prefix or suffix computation has its required data on its own side of a true cut. Later main data for any guessed role, and later buffer data for any decoded offset, are scheduled after the suffix start.*
- (iv) *Every possible true first-episode end is after all these cuts and bounded control windows.*

Proof. Three finite envelopes. First translate the fixed buffer arrangements intact so that

$$s < B_{1,0} < a_1 < B_{0,0} < a_0$$

with each buffer's own boundaries and cut offsets in their prescribed order. Each difference $B_{i,j} - s$ or $a_i - s$ is a fixed offset. Set

$$D = \text{diam } \mathcal{H} + h_0 + h_1 + w + 1.$$

The main hypotheses differ from the true origin by at most $\text{diam } \mathcal{H}$. Aligned hypotheses differ by $p - p'$, of magnitude at most h_i ; stage 1 adds γ of magnitude at most h_0 , and an A_1 test adds at most w . Thus D bounds all the end-system displacements that will be used.

Include in an inner envelope every inner search, inspection block, boundary, and main cut at origins displaced by at most D , together with the additional shifts of magnitude at most $\max_r R_r$ needed for usability. Every such position lies in $[t - C, t + C]$ for a finite constant $C \geq D + 1$ fixed before L is chosen. The marker radii and the bounded phase displacements used to define the B_j are included in C . Choose L so that $L - C > a_0 - s$.

Likewise, include all end searches, their marker-inspection windows, and their seeds at displacements of magnitude at most D . They lie in $[z_e(t) - C', z_e(t) + C']$ for a finite constant C' independent of o_e ; increase it so that $C' > d_e + D$. Choose o_e so that $o_e - C' > C$. Finally choose

$$b_{\text{tail}} > C' + d_e + 1.$$

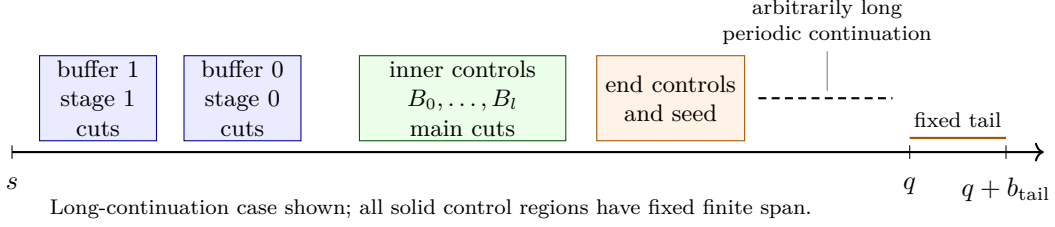


Figure 1: The order of the computation regions in one episode, not to scale. A marker-absent episode with a long periodic continuation is shown. The control regions include bounded windows for translated hypotheses. With an earlier first mismatch, end-control windows may extend into the fixed tail; all remain before the episode endpoint $q + b_{\text{tail}}$.

Any anchor satisfies $q \geq z_e(t) - d_e$. Therefore $q + b_{\text{tail}} > z_e(t) + C' + 1$, proving that the true end is beyond the whole end envelope, and hence beyond both earlier envelopes.

Data on the required side of each cut. It remains to verify the sidedness of the computations. At a main cut $k = t + c_j + \delta_r a$, the spacing of the main centers puts all inspections for boundaries through B_{j-1} before k . For any guessed role (j, r) in a suffix and any corresponding hypothesis $t' = k - c_j - \delta_r a$, its first required center is

$$z_j(t') = k + G/2 - \delta_r a.$$

Since $0 \leq \delta_r a < R_r$ and $G > 10(H + r_o + \max_r R_r + 1)$, its inspection windows, boundary, and all later main data lie after k . This calculation holds independently of the role that produced the actual prefix cut.

For a buffer hypothesis p' of interval kind j , the hypothetical start is $s' = k - p'$. The first later boundary has offset $(B_{i,j} - s) - p' > 0$ from k , and all required preceding data lie on the prefix side when the hypothesis is true. The order 1,0 puts all stage-0 segment data after every stage-1 cut; hence the stage-1 suffix can form its later terminal table. All inner and end data are later still by the envelope choices. This proves every assertion. $\square$

All the choices in this proof are acyclic. The inner construction fixes $\mathcal{P}_0$ and $\mathcal{H}$ before w is chosen; then $\mathcal{P}_1, d_e, K_e, r_e$ are fixed. Translating the buffers leaves their differences unchanged. Only after that do we choose L , then o_e , then b_{tail} . None of these last choices changes an ambiguity bound.

Definition 5.3. Let E consist of the nonempty words starting at s for which the required data are available, the anchor rule defines q , and the word ends exactly at $q + b_{\text{tail}}$. The rule is interpreted using the placement just fixed. Since all tests translate with the letters, it does not depend on an ambient coordinate origin.

Lemma 5.4 (Prefix code). *The episode language E is a prefix code: no member is a proper prefix of another member. All marker and bounded control decisions associated with an episode are unchanged when letters are appended.*

Proof. By Lemma 5.2, each decision window is internal before the prescribed endpoint. Its letters therefore remain unchanged under extension. A present-marker anchor is unchanged. In the absent case, the seed and its periodic extension are unchanged, and the first mismatch already seen remains the first mismatch. Thus any extension has the same prescribed endpoint. It can be a complete episode only if it ends there. $\square$

5.3 A guard that forces the correct end

An *exact-end test at an origin* applies the anchor rule at that origin and requires the entire test argument to end at $q + b_{\text{tail}}$. In particular it includes the mismatch letter when the marker is absent. A suffix may make this test at a hypothesized origin preceding its own start: it must still possess all the actual seed and inspection windows required by that hypothesis.

For a fixed finite nonempty list of candidate origins, define its *end guard* to require equal values of η at all candidates and to apply the exact-end test at one fixed reference candidate. The list and reference are fixed by the suffix test's finite choices. All candidate windows are mandatory under the convention above.

Lemma 5.5 (Exact end guard). *Suppose a finite word begins with a complete episode e of true origin t and anchor q . A prefix cut k lies inside this first episode. Let a suffix argument starting at k use an end guard whose candidate list contains t and has diameter at most w . If the guard accepts, the suffix ends exactly at the end of e , and its reported anchor is q . This conclusion holds even if the proposed suffix initially ends before e or extends past e into later letters.*

On the actual suffix of e , the guard accepts whenever $A_1(t)$ is false, provided the placement conditions for its list hold.

Proof. Agreement of the actual anchors. An accepted argument contains every mandatory window, so all the candidate marker decisions agree with the corresponding decisions in the containing word. The true candidate therefore has its true value of η . If the common value is a present marker, the reference's anchor is immediately q , and the exact-end test forces the claimed end.

Suppose instead that the common value is $\perp$. Every candidate seed has a period less than d_e . Their starts differ by at most w , and

$$K_e - w > d_e^2.$$

Their seeds consequently determine the same two-sided periodic word by Lemma 4.2. The overlapping union of the seeds consists entirely of letters of this word. The true first mismatch is after the true seed. Every later candidate seed starts before the true seed ends, since its displacement is at most $w < K_e$. The mismatch cannot precede the start of such a seed, and it cannot lie in any later candidate seed: that seed consists of actual letters of the same periodic word, whereas the mismatch letter differs from it. A candidate seed starting earlier also ends before the true seed ends. Thus the true first mismatch follows the entire seed union. It is therefore also the first mismatch after the reference seed.

Rejection of premature and excessive consumption. For a proposed suffix ending too soon, either a mandatory window is absent, or no earlier reference mismatch exists to certify its proposed endpoint. For a suffix extending too far, the actual first mismatch is still the first one, so its exact-end test still requires the original endpoint $q + b_{\text{tail}}$. Both cases are excluded.

Acceptance on a stable true episode. Finally, if $A_1(t)$ is false, every candidate is within distance w of t and has the same η value. On the true suffix all windows are available by placement. The argument above shows that the reference exact-end rule supplies the true endpoint, so the guard accepts. $\square$

Only the reference candidate makes an unbounded continuation search in this guard. The other candidates supply bounded marker and seed data. At the last residual stage we will instead decode the origin from bounded data before making its one exact-end test. These distinctions will also be used to compile the suffix tests at height one.

6 Three split constructions

All timing data are now fixed. We use them to define three deterministic episode updates and their prefix/suffix tests. The main update has the true monoid action as its linear part. Two larger affine updates encode the main update and then each other on distinguished basis vectors. Their graph languages are constructed in reverse order, starting from an empty skipped class. This section proves semantic correctness of those tests; Section 7 establishes their expression heights.

There are three different orders to keep separate. Along the word, buffer 1 precedes buffer 0 and the main segment. The total update tables are defined in the order g_0, g_1, g_2 . The graph tests are constructed by stage 1, stage 0, and the main split. Every episode update is fixed on the entire episode before a split is selected.

Throughout this section, an unavailable inspection window or an unordered or unavailable product interval makes a test fail. In particular, every window belonging to a candidate list is mandatory: a test never removes an unavailable candidate and then checks uniqueness among those remaining. The placement in Lemma 5.2 supplies these windows on all the true uses below, including the windows at false candidate origins.

6.1 The main update and split

Fix an episode $e \in E$, with start s , origin $t = s + L$, anchor q , and end $b = q + b_{\text{tail}}$. Write

$$T_e = T_{s,b}, \quad d_j = T_{B_{j-1}, B_j} \quad (1 \leq j \leq l), \quad \mathbf{d} = (d_1, \dots, d_l).$$

Recall that, for a residue $\ell \in \mathbb{Z}/n\mathbb{Z}$,

$$\mathcal{E}(\ell) = \{(\alpha, \lambda) : \ell \in I_\alpha + J_\lambda\}$$

is its clock edge set. By Lemma 3.1, this set is either contained in the diagonal, possibly empty, or consists of one off-diagonal edge. With $\ell = q - t \pmod{n}$, define $g_{0,e} : V \rightarrow V$ by

$$g_{0,e}(x) = \begin{cases} xT_e + \beta(\mathbf{d})T_{B_l,b}, & \mathcal{E}(\ell) \text{ is contained in the diagonal,} \\ xT_e + Y(\lambda) - X(\alpha)T_e, & \mathcal{E}(\ell) = \{(\alpha, \lambda)\}, \quad \alpha \neq \lambda. \end{cases} \quad (6.1)$$

Here $X(\alpha)$ and $Y(\alpha)$ denote the corresponding fields of a tag. In both cases this is an affine map with linear part $x \mapsto xT_e$. All terms in its definition are fixed by the episode.

We define languages $P_x^{\text{main}}, Q_y^{\text{main}}$ for $x, y \in V$. In specifying a prefix test, s is the start of its argument; in specifying a suffix test, k is the start of its argument. These coordinates are relative to the argument and do not require an external position origin.

The prefix test P_x^{main} chooses a tag

$$\alpha = (j, r, v, d_1, \dots, d_{j-1}, X, Y)$$

and an integer $0 \leq a < n$. Its argument must end at

$$k = t + c_j + \delta_r a, \quad t = s + L.$$

It requires

$$k - t \pmod{n} \in I_\alpha, \quad x = X, \quad xT_{s,B_0} = v,$$

and checks the claimed products $d_f = T_{B_{f-1}, B_f}$ for $f < j$. Only boundaries and inspections preceding its cut are used.

The suffix test Q_y^{main} chooses its own tag λ , independently of the prefix choice. First it applies the end guard of Lemma 5.5 to the entire list

$$\{k - h : h \in \mathcal{H}\}.$$

Thus it requires a common value of η at these origins and applies the exact end rule at a fixed reference origin in the list. Denote the resulting anchor by q and the argument end by $b = q + b_{\text{tail}}$. The test requires

$$q - k \pmod{n} \in J_\lambda, \quad y = Y(\lambda).$$

Write the selected tag as $\lambda = (j, r, v, d_1, \dots, d_{j-1}, X, Y)$. For *every* origin

$$\tau = k - c_j - \delta_r a, \quad 0 \leq a < n,$$

the test computes $B_j(\tau), \dots, B_l(\tau)$ by the schedule at τ , and supplies the later products

$$d_f^{(\tau)} = T_{B_{f-1}(\tau), B_f(\tau)} \quad (j < f \leq l).$$

It checks that the following function of $c \in M$ is constant:

$$c \mapsto v(d_1 \cdots d_{j-1} c d_{j+1}^{(\tau)} \cdots d_l^{(\tau)}) + \beta(d_1, \dots, d_{j-1}, c, d_{j+1}^{(\tau)}, \dots, d_l^{(\tau)}). \quad (6.2)$$

If its constant value is z_τ , it further requires

$$z_\tau T_{B_l(\tau), b} = y.$$

Empty product lists have their usual identity interpretation. All these checks are made within the suffix argument. In particular, the exact end rule is run only at the reference origin; other origins use its fixed anchor q and bounded controls.

Two different lists occur in this definition. The end guard uses every $h \in \mathcal{H}$, independently of the chosen tag, so it contains the true origin even when the suffix chooses a wrong role. The line-constancy tests use all n origins for the suffix's own role (j, r) . Both lists are fixed and retain all their mandatory windows. The first enforces a true end; the second will use product transport for completeness.

The main split skips episodes whose clock, inner transport, or end stability may fail. Its residual decoder will use the predicate A_0 , but first it must obtain a common anchor from the end guard. For that decoder we require A_1 to be false at every candidate origin, uniformly over all choices of the cut. These origins are $t + p - p'$ with $p, p' \in \mathcal{P}_0$, so their displacements from t form Δ_0 . This requirement determines the second exceptional class. Set

$$\begin{aligned} D_0 &= \{e \in E : A_0(t, q) \text{ or } A_1(t)\}, \\ D_1 &= \{e \in D_0 : \text{there exists } \gamma \in \Delta_0 \text{ with } A_1(t + \gamma)\}, \\ D_2 &= \emptyset. \end{aligned} \quad (6.3)$$

Thus D_1 consists of those episodes already in D_0 for which the all-candidate end-stability requirement fails. Since $0 \in \Delta_0$, an episode in $D_0 \setminus D_1$ satisfies $A_0(t, q)$ and has A_1 false at every $t + \gamma$, $\gamma \in \Delta_0$. Stage 0 can therefore use a common anchor and seek the true origin through A_0 . Stage 1 will instead decode the bounded end-instability predicate in the definition of D_1 before testing that origin's end.

The buffers will implement affine lifts $g_1 : U_1 \rightarrow U_1$ and $g_2 : U_2 \rightarrow U_2$, defined below. The three planned split applications are

Split	Update tested	Episode class	Skipped class
main	g_0	E	D_0
stage 0	g_1	D_0	D_1
stage 1	g_2	D_1	$D_2 = \emptyset$

These applications are used in the order stage 1, stage 0, main.

Proposition 6.1 (Main split). *The languages $P_x^{\text{main}}, Q_y^{\text{main}}$ satisfy the two split hypotheses of Lemma 2.1 for the episode update g_0 , with $D = E$ and $D' = D_0$.*

Proof. Soundness for arbitrary independent tags. Suppose that a prefix of a word in E^* factors as $P_x^{\text{main}}Q_y^{\text{main}}$. The prefix length is positive and its cut lies before the earliest possible first episode end, by Lemma 5.2. Hence there is a first episode e . If the prefix chose $k = t + h$ with $h \in \mathcal{H}$, then the true origin t belongs to the suffix's end-guard list. Its diameter is $\text{diam } \mathcal{H} < w$. Lemma 5.5 therefore forces the concatenation to end exactly at the end of e , with its true anchor q . This conclusion applies also to a proposed factor ending too early or extending into later episodes; it does not presume that the suffix has already identified the episode correctly.

Let α and λ be the independently chosen prefix and suffix tags. The two residue tests give $(\alpha, \lambda) \in \mathcal{E}(q - t)$. If this edge set is contained in the diagonal, then $\alpha = \lambda$. The true origin is consequently among the candidates for the selected role (j, r) in the suffix test. At that candidate, the transmitted vector and preceding products are correct by the prefix checks, and the later products are correct by the suffix checks. Evaluating the constant function (6.2) at the actual d_j gives the true corrected vector at B_l . Its transport to b proves $g_{0,e}(x) = y$. If instead there is one off-diagonal edge, the prefix forces $x = X(\alpha)$ and the suffix forces $y = Y(\lambda)$. Formula (6.1) again gives $g_{0,e}(x) = y$. Thus every accepted split is sound, regardless of which tags were guessed.

Completeness at one usable scale. Let $e \in E \setminus D_0$ and fix $x \in V$. Then $q - t \pmod{n} \notin W$, some scale r is usable at t , and $A_1(t)$ fails. Put $v = xT_{s,B_0}$. Lemma 2.3 chooses an interval j for which the corrected segment function is constant on the coordinate line through the true tuple $\mathbf{d}$. Choose the tag with this j, r, v , the true preceding products, and

$$X = x, \quad Y = g_{0,e}(x).$$

The full diagonal is available outside W . Since $\gcd(\delta_r, n) = 1$, the cuts $t + c_j + \delta_r a$ for $0 \leq a < n$ realize every residue modulo n . One therefore satisfies both clock tests for this tag.

The end guard succeeds because $A_1(t)$ fails. Each candidate for the selected role differs from t by a multiple of δ_r of magnitude at most R_r . Lemma 4.4 leaves all its later interval products and its B_l -to- b product unchanged. Thus every candidate in the universal suffix check has the same constant function and the same final output as the true candidate. Notice that this uses only the later products: no claim about the vector at a shifted B_0 is needed. The selected cut gives a split with output $g_{0,e}(x)$. $\square$

6.2 Total tables and affine lifts

To handle D_0 , we encode the entire main update, including its affine correction, in the linear part of an update on U_1 . Repeating this construction on U_2 will handle the second exceptional class. The notation $[v, c]$ always denotes a basis vector of $U_{i+1} = \mathbf{F}_2^{U_i \times M}$, even when $v = 0$; in particular, $[0, 1_M]$ is distinct from the zero vector of U_{i+1} .

For each episode, a *total prefix table* at a_i is a family of maps

$$\Phi_i(c) : U_i \longrightarrow U_i \quad (c \in M)$$

determined by the anchored schedule and the suffix data from a_i onward, such that $\Phi_i(T_{s,a_i}) = g_{i,e}$. The table must be defined at every $c \in M$, including values that are not attainable by a prefix of the relevant length.

Here is the initial table explicitly. Set

$$A = T_{a_0,b}, \quad C = T_{B_l,b}.$$

The main tuple $\mathbf{d}$ and the clock edge set are determined from the suffix after a_0 . For every $c \in M$ and $v \in U_0 = V$, define

$$\Phi_0(c)(v) = \begin{cases} v(cA) + \beta(\mathbf{d})C, & \mathcal{E}(q-t) \text{ is contained in the diagonal,} \\ v(cA) + Y(\lambda) - X(\alpha)(cA), & \mathcal{E}(q-t) = \{(\alpha, \lambda)\}, \quad \alpha \neq \lambda. \end{cases} \quad (6.4)$$

All products retain their reading order. Substitution of $c = T_{s,a_0}$ gives (6.1), while the formula itself uses no knowledge of that true prefix product.

Lemma 6.2 (Total tables and affine lifts). *There are episode updates $g_{1,e} : U_1 \rightarrow U_1$ and $g_{2,e} : U_2 \rightarrow U_2$, affine for every $e \in E$, with the following properties. For $i = 0, 1$, the update $g_{i,e}$ has a total prefix table Φ_i at a_i . Define a linear map $H_i : U_{i+1} \rightarrow U_{i+1}$ on basis vectors by*

$$[v, c]H_i = [\Phi_i(c)(v), 1_M].$$

Writing $\mathbf{d}_i = (T_{B_{i,0},B_{i,1}}, \dots, T_{B_{i,l_i-1},B_{i,l_i}})$, the next update is

$$g_{i+1,e}(z) = \left(z\rho_i(T_{s,a_i}) + \beta_i(\mathbf{d}_i)\rho_i(T_{B_{i,l_i},a_i}) \right) H_i. \quad (6.5)$$

Its linear part $L_{i+1,e}$ satisfies

$$L_{i+1,e} = \rho_i(T_{s,a_i})H_i, \quad [v, 1_M]L_{i+1,e} = [g_{i,e}(v), 1_M] \quad (v \in U_i). \quad (6.6)$$

Proof. The initial total table is (6.4). Given any total table Φ_i , prescribing H_i on the basis defines a unique linear map. The maps $\Phi_i(c)$ need not be linear as maps on U_i : their values only select the images of basis vectors in the larger space. Formula (6.5) is therefore affine, with the linear part displayed in (6.6). Moreover,

$$[v, 1_M]\rho_i(T_{s,a_i})H_i = [v, T_{s,a_i}]H_i = [\Phi_i(T_{s,a_i})(v), 1_M] = [g_{i,e}(v), 1_M].$$

The lift has been constructed from a total table. We must still verify that the next suffix can compute such a table without the earlier prefix. This is the precise role of the reversed physical buffer order.

It remains to obtain the table Φ_1 needed to construct g_2 . The buffer order is 1 then 0, so a_1 precedes all the segment data of buffer 0. Define, for every $c \in M$ and $z \in U_1$,

$$\Phi_1(c)(z) = \left(z\rho_0(cT_{a_1,a_0}) + \beta_0(\mathbf{d}_0)\rho_0(T_{B_{0,l_0},a_0}) \right) H_0. \quad (6.7)$$

The actual tuple $\mathbf{d}_0$, the displayed suffix products, and the entire map H_0 are available from the suffix after a_1 . Indeed, H_0 is built from the total table Φ_0 , whose values depend only on the later anchored schedule and suffix products; it does not require the true value of T_{s,a_0} . More explicitly, with $q, t, \mathbf{d}, A, C$ fixed by those later data, the suffix enumerates (6.4) for every $c' \in M$ and $v \in U_0$. Those values give every basis image of H_0 , including images indexed by infeasible prefix values. Thus (6.7) is a total table with the required dependence. Substitution of $c = T_{s,a_1}$ yields (6.5) for g_1 . The preceding construction then gives H_1 and g_2 . All these maps are defined from the complete episode independently of any split or skipped class. $\square$

Lemma 6.3 (Recovery on sequences). *For any $D \subseteq E$ and $i \in \{0, 1\}$, graph tests for g_{i+1} on D^* yield graph tests for g_i on D^* by finite Boolean operations. Consequently this recovery does not increase generalized star height.*

Proof. For a sequence $e_1 \cdots e_k \in D^*$, the linear part of the affine composite of $g_{i+1, e_1}, \dots, g_{i+1, e_k}$ is $L_{i+1, e_1} \cdots L_{i+1, e_k}$, in reading order. Equation (6.6) shows that the distinguished basis vectors return to distinguished basis vectors after every episode. By induction, this product sends $[v, 1_M]$ to

$$[g_{i, e_k}(\cdots g_{i, e_1}(v) \cdots), 1_M].$$

If $R_{\xi, \zeta}$ are graph tests for the affine composite on U_{i+1} , the graph test for sending $v \in U_i$ to $w \in U_i$ is therefore

$$\bigcup_{f \in U_{i+1}} \left(R_{[v, 1_M], [w, 1_M] + f} \cap R_{0, f} \right),$$

by Lemma 2.2. The zero in $R_{0, f}$ is the zero vector of U_{i+1} . This formula applies to the entire sequence, including the empty sequence, and proves the claim. $\square$

6.3 Two aligned decoders

We have obtained affine updates and a recovery identity for entire sequences. It remains to give the two residual splits. Their algebraic transmission rule is the same; their origin decoders use different predicates and a different order of end detection.

For $i = 0, 1$, we now define $P_x^{(i)}, Q_y^{(i)}$ for inputs and outputs $x, y \in U_{i+1}$. These tests implement g_{i+1} on D_i , with skipped class D_{i+1} .

The prefix test chooses an offset $p \in \mathcal{P}_i$ and ends at $k = s + p$. If the kind assigned to p is

$$(j, v, d_1, \dots, d_{j-1}),$$

it checks

$$v = x\rho_i(T_{s, B_{i,0}}), \quad d_f = T_{B_{i,f-1}, B_{i,f}} \quad (1 \leq f < j).$$

The relevant boundaries are the fixed buffer positions relative to s , and all these checks precede the cut.

The suffix lists the candidate origins

$$\tau_{p'} = k + L - p' \quad (p' \in \mathcal{P}_i).$$

Its decoding and end checks are as follows.

- (1) At stage 0, require $A_1(\tau_{p'})$ to be false for every candidate. Apply the end guard to the full candidate list and obtain its anchor q . Require a unique p' such that $A_0(\tau_{p'}, q)$ holds. All evaluations of A_0 use this same anchor; they make no additional unbounded end searches.
- (2) At stage 1, first require a unique p' such that

$$\text{there exists } \gamma \in \Delta_0 \text{ with } A_1(\tau_{p'} + \gamma).$$

This test uses bounded controls and does not involve q . After decoding p' , apply the exact end rule at $\tau_{p'}$ to obtain q and the argument end.

At stage 0 the candidate list has diameter $h_0 < w$, as required by the end guard. Stage 1 does not use that guard and needs no such bound on h_1 .

After decoding, set $s' = k - p'$ and use the kind attached to p' , say $(j, v, d_1, \dots, d_{j-1})$. All subsequent buffer positions are scheduled relative to s' , and the late controls use $t' = s' + L$ and the anchor just obtained. The suffix supplies the later products d_f for $j < f \leq l_i$ and checks that

$$c \mapsto v\rho_i(d_1 \cdots d_{j-1} c d_{j+1} \cdots d_{l_i}) + \beta_i(d_1, \dots, d_{j-1}, c, d_{j+1}, \dots, d_{l_i}) \quad (c \in M)$$

is constant. If its value is z , the test requires

$$(z\rho_i(T_{B_{i,l_i},a_i}))H_i = y.$$

The map H_i is computed from the explicit total tables (6.4) and (6.7), keeping the decoded time and anchor data fixed. These tables call no graph test on an episode sequence. The suffix uses only its own data; if an incorrect hypothesis places required data outside its argument, the test rejects that argument.

Proposition 6.4 (Aligned splits). *For $i = 0, 1$, the languages $P_x^{(i)}, Q_y^{(i)}$ satisfy the two hypotheses of Lemma 2.1 for the update g_{i+1} , with $D = D_i$ and $D' = D_{i+1}$.*

Proof. Soundness: decode the true offset and force the true end. Consider an accepted concatenation $P_x^{(i)}Q_y^{(i)}$ at the start of a word in D_i^* . The positive prefix cut lies before every possible first episode end, so there is a first episode $e \in D_i$. If the prefix chose p , then its true origin occurs in the suffix list as

$$t = s + L = k + L - p = \tau_p.$$

At stage 0, the end guard therefore forces exact consumption of this first episode and supplies its true q . The suffix also requires $A_1(t)$ to fail. Since $e \in D_0$, this implies $A_0(t, q)$. The true candidate p thus satisfies the decoding predicate, so uniqueness forces the decoded p' to equal p .

At stage 1, membership of e in D_1 directly implies that p satisfies the decoding predicate. Every window in that predicate is mandatory and, on the true episode, lies inside the episode. An accepted proposed suffix therefore sees the same bounded data even if its proposed end was too early or too late. Uniqueness first forces $p' = p$; the exact end rule at this correctly decoded origin then forces exact consumption of e .

In both stages the prefix and suffix have consequently used the same kind and the true buffer schedule. The prefix supplies the correct vector at $B_{i,0}$ and the preceding interval products; the suffix supplies the correct later products. Line constancy then gives the true corrected vector at B_{i,l_i} . Its subsequent work action and terminal map H_i give exactly (6.5), proving $g_{i+1,e}(x) = y$.

Completeness: count all lost offsets of the needed kind. Fix $e \in D_i \setminus D_{i+1}$ and $x \in U_{i+1}$. Apply Lemma 2.3 to the true buffer tuple and the vector

$$v = x\rho_i(T_{s,B_{i,0}}).$$

It gives an interval j and thus one required kind $(j, v, d_1, \dots, d_{j-1})$. Every offset of that kind passes the prefix checks. There are more than C_i such offsets.

At stage 0, the condition $e \notin D_1$ makes A_1 false at all $t + \gamma$, $\gamma \in \Delta_0$. For every cut p , all candidate origins are of this form, so the absence checks and the stable end guard pass and give the true q . Moreover $A_0(t, q)$ holds, so the true candidate passes the current decoding predicate. If an offset p fails uniqueness, some $p' \neq p$ satisfies $A_0(t + p - p', q)$. By Proposition 4.5, at most C_0 ordered pairs (p, p') have this property. In particular at most C_0 offsets p can fail uniqueness.

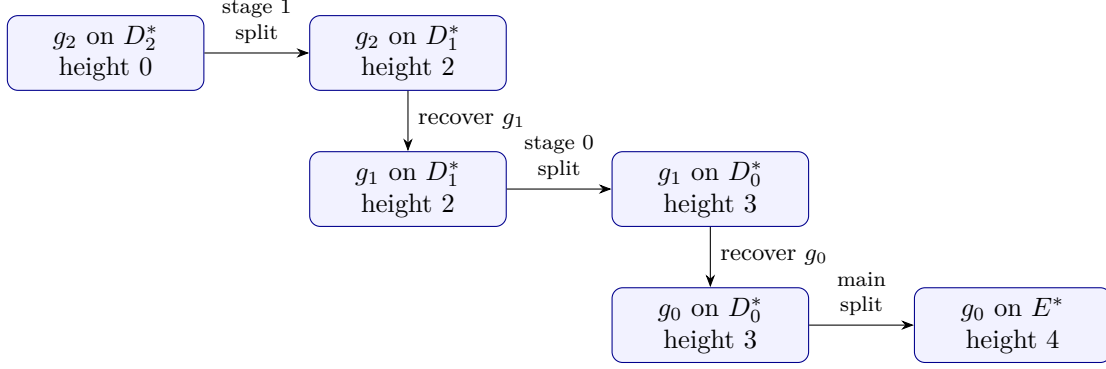


Figure 2: Construction of the sequence graph languages. Horizontal arrows apply a split; vertical arrows recover the lower update by finite Boolean operations. The labels 0, 2, 3, 4 are the height bounds proved in Section 7. Each recovery uses the same word in both of its affine tests.

At stage 1, true membership in D_1 supplies the true candidate for every cut. A failure of uniqueness gives an ordered pair $p \neq p'$ with

$$\text{there exists } \gamma \in \Delta_0 \text{ with } A_1(t + p - p' + \gamma).$$

Lemma 5.1 bounds the number of these pairs, and hence the number of lost offsets, by C_1 .

In either stage, a copy of the required kind remains. At its correctly decoded origin all later data are available by placement, and the affine-line check gives the required output $g_{i+1,e}(x)$. This proves completeness. $\square$

6.4 The recovery chain

The input and output of each split are now established. Figure 2 shows how the graph languages are assembled. A Boolean recovery changes which update is being tested while leaving the episode domain and the height bound fixed. A split enlarges that domain and is the only operation in this chain that introduces an additional star.

Proposition 6.5. *Three applications of Lemma 2.1, with intervening finite Boolean operations, construct tests for the T -value on E^* from the component languages defined above.*

Proof. On $D_2^* = \{\epsilon\}$, the graph tests for g_2 are $\{\epsilon\}$ for equal input and output and empty otherwise. Apply Proposition 6.4 at stage 1 and Lemma 2.1 to obtain graph tests for g_2 on D_1^* . Lemma 6.3 converts these to graph tests for g_1 on D_1^* .

Use those tests as the skipped-class tests in the stage-0 application of Proposition 6.4. The split lemma gives graph tests for g_1 on D_0^* , and Lemma 6.3 recovers graph tests for g_0 on D_0^* . These are the skipped-class tests required by Proposition 6.1, whose application yields graph tests for g_0 on E^* .

Finally, Lemma 2.2 recovers the product of the linear parts of the maps $g_{0,e}$. By (6.1), these are the true T_e actions on V . Applying their product to the basis vector indexed by 1_M identifies the T -value of the whole episode sequence. Only the three stated split applications introduce stars through the split formula; all recovery operations are finite Boolean operations. Section 7 will establish the height bounds for the component languages and complete the count. $\square$

7 Compilation over the original alphabet

The semantic construction is complete: three splits and Boolean recoveries determine the monoid value on episode sequences. To turn this into a height-four expression, we must account for the component languages and the final remainder. This section proves both claims directly over Σ . In particular, a suffix language must have its own height-one expression even when its guessed origin is wrong and no matching prefix exists.

The common mechanism is a finite collection of literal word templates ab^hc . Every component has at most one unbounded continuation; it follows one fixed period until its first mismatch. On a fixed template, all other tests become ultimately periodic conditions on the exponent h . This provides the expression itself, rather than inferring a height bound from mere regularity.

7.1 A template lemma

Lemma 7.1 (Periodic templates). *Consider a finite collection of templates*

$$ab^hc, \quad h \geq 0, \tag{7.1}$$

where $a, b, c \in \Sigma^*$ are fixed for each template and $b \neq \epsilon$. Suppose a test on each template uses only the following data:

- (i) letters in fixed finite collections of windows at offsets from the argument's start or end, with mandatory availability checks;
- (ii) order and availability of endpoints chosen from fixed finite sets of offsets from the start or end;
- (iii) M -products between these endpoints, whenever available and ordered, and lengths modulo fixed positive integers;
- (iv) fixed finite combinations, choices, and calculations from these data.

The accepted words have a generalized expression of height at most one. Adding or removing a finite language preserves this bound.

Moreover, any of the exact-end tests of Section 5, with its reference or decoded origin chosen from finitely many offsets relative to the test start, is covered by a finite language and finitely many templates of the form (7.1).

Proof. Ultimately periodic data and explicit expressions. Fix a template. For sufficiently large h , all the indicated availability and order relations stabilize. Letters at a bounded distance from the start or end also stabilize, since increasing h inserts a full copy of b . A product between boundedly separated endpoints then stabilizes. A product spanning the growing middle has the form

$$uT(b)^{h-j}v$$

for fixed $u, v \in M$ and a fixed integer j , once h is sufficiently large. This follows by separating its fixed initial and terminal pieces from its complete copies of b . Powers of an element of a finite monoid are ultimately periodic: if two powers coincide, multiplying by further powers gives the eventual period. Lengths modulo any fixed integer are periodic in h . Thus the entire finite data array is ultimately periodic. A fixed finite computation from this array, including a universal or uniqueness check on a fixed finite list, remains ultimately periodic.

Let the resulting accepted exponents have period $f > 0$ after a threshold. They are a finite set together with finitely many progressions $h_0 + f\mathbb{N}$. Each progression contributes the expression

$$ab^{h_0}(b^f)^*c,$$

where the displayed fixed powers are finite words. These expressions have height one; the exceptional words have height zero. A finite union proves the first assertion, including a finite number of templates and finite exceptions.

The template cover for an independent exact-end test. For the final assertion, fix one possible end-rule origin relative to the argument start. The present-marker branch has a bounded anchor and therefore bounded total length. For the absent-marker branch, the seed is at a fixed offset and has fixed length. Make finite choices of all letters through the seed, of its short period and phase, and of the fixed-length tail beginning at the first mismatch. These choices fix a prefix a through the seed and a nonempty period word b beginning at the next position. If the mismatch occurs after h full copies of b and a partial copy of length $r < |b|$, place that partial copy, the mismatching letter, and its remaining $b_{\text{tail}} - 1$ letters in c . There are finitely many such choices, and the mismatching letter is required to differ from the next periodic letter. They give exactly templates (7.1) for this continuation and end. The bounded marker and availability conditions can still be tested on the templates. Taking the finite union over possible end-rule origins proves the assertion. $\square$

This proof applies to the exact-end rule of a suffix considered by itself. It does not presuppose that its guessed origin is correct, or that it is actually the suffix of an episode. If a hypothesized origin lacks a mandatory seed or window, it is rejected. Otherwise its own exact-end rule supplies the template.

Proposition 7.2 (Component bounds). *The episode languages E, D_0, D_1 and every suffix component Q_y in Section 6 have generalized star height at most one. Every prefix component P_x has height zero. These bounds hold over the original alphabet Σ .*

Proof. We first identify the single end rule that supplies each template cover, then verify that every remaining operation is a finite calculation from the allowed data. No part of this argument assumes that a standalone suffix has guessed the true episode origin.

All prefix cuts come from fixed finite sets of offsets, so the lengths of P_x words are bounded. Hence P_x is finite.

Every other language in the assertion has one exact-end rule on its entire argument. For E, D_0, D_1 this is the true episode rule. A main or stage-0 suffix uses the reference rule in its end guard. A stage-1 suffix uses the rule at its uniquely decoded candidate. For a stage-1 suffix considered alone, first make a finite case distinction for the decoded offset p' . The uniqueness predicate inspects all candidates' bounded end-search windows and their finitely many γ translates; it contains no mismatch search. The case then has just the exact-end rule at $k + L - p'$, whether or not that is a genuine episode origin. There are only finitely many reference or decoded offsets; we may take a finite union over them. Lemma 7.1 therefore supplies the finite template cover even for standalone suffixes with incorrect hypotheses.

We check that the remaining tests use only the finite data in that lemma. Marker searches and their stability checks inspect bounded windows at fixed nominal offsets. So do usability, the phase and lattice choices, and all additional candidate translates. Every selected main boundary lies within a fixed bounded neighborhood of its center. Buffer boundaries have fixed offsets. Consequently all partial products in the component computations have endpoints selected from finite sets of start offsets, except for the argument end itself.

The anchor supplied by the exact-end rule is the argument end minus b_{tail} . Thus the clock and A_0 use only fixed-modulus differences between this end offset and their candidate origins. The predicates A_1 and the stage-1 decoding predicate are wholly bounded. Equality, universal checks, and uniqueness over their finite candidate lists are finite calculations, with the mandatory availability convention retained for every candidate.

The affine-line checks range over finite vector spaces and the finite monoid M . Their data consist of the specified states and actual partial products just described. The tables Φ_i and H_i are computed by their explicit formulas using hypothetical elements of M and actual suffix products and schedule data. Their finite recursion does not invoke a graph test on an episode sequence. The total-table property permits every hypothetical monoid value, so no extra prefix existence test is being imposed.

Finally, no translated candidate makes a second unbounded continuation search. The guards compare bounded marker data and use just their one reference rule. Stage 1 makes bounded decoding checks before applying the one decoded rule. Accordingly all remaining acceptance conditions are fixed finite calculations from the template data. Applying Lemma 7.1 proves the bounds. Every resulting expression uses finite words in Σ^* and Boolean operations, concatenation, and stars over Σ ; no auxiliary tag has become an input letter. $\square$

7.2 The final remainder

The episode rule does not require a word to end in a complete episode. Define

$$F = \neg(E\Sigma^*), \quad \Sigma^* = \neg 0.$$

Thus F consists of words with no episode prefix and has height at most one. To finish, we need the same bound for its monoid-value tests.

Lemma 7.3 (Remainder). *For every $d \in M$, the language $F \cap T^{-1}(d)$ has height at most one. Every word admits a unique factorization into a sequence of episodes followed by a word in F .*

Proof. Product tests on the incomplete tail. Choose a fixed length threshold exceeding all the bounded decision windows at a first start and all possible first-episode lengths in the present-marker case. A longer word in F cannot have a present end marker: that would already give an episode prefix. Its marker-absent seed has a fixed short-period continuation. Either no mismatch has yet occurred, or its first mismatch is at a position q satisfying

$$|u| - q < b_{\text{tail}}$$

when the word u starts at position zero. Otherwise the prefix ending at $q + b_{\text{tail}}$ is a complete episode. All the required bounded windows for that prefix are internal by Lemma 5.2, even when the mismatch itself is early.

In the no-mismatch case, finite choices of the bounded initial word, period, phase, and final partial period give templates ab^hc . In the mismatch case, append one of the finitely many possible post-mismatch portions of length less than b_{tail} ; these again give finitely many such templates. Words below the threshold form a finite language. On every template the test $T(u) = d$ is an ultimately periodic monoid-product test, so Lemma 7.1 gives a height-one language with that value on the template cover. Intersecting with F gives exactly $F \cap T^{-1}(d)$ and keeps the same bound.

Unique parsing. Repeatedly remove an episode prefix while one exists. Every episode is nonempty, so this procedure terminates, and the remaining word is in F . Lemma 5.4 makes each removed prefix unique. Any alternative E^*F factorization must therefore remove the same successive episodes; it also cannot stop earlier while an episode prefix remains. This proves uniqueness. $\square$

7.3 The height bound

Proof of Theorem 1.1. The component bounds needed for the three applications of Lemma 2.1 are supplied by Proposition 7.2. If the skip graph tests have height at most b , that lemma gives the

new bound

$$\max\{2, b + 1\}.$$

Indeed the episode-domain test D^* has height at most two, and the single star in the split formula lies over component and skip tests of height at most $\max\{1, b\}$.

The stage-1 skipped class is $D_2 = \emptyset$, whose graph tests consist only of the empty-sequence tests and have height zero. In the order proved in Proposition 6.4 and Proposition 6.1, the bounds are therefore

$$\underbrace{0}_{D_2^*} \longrightarrow \underbrace{2}_{\text{stage 1}} \longrightarrow \underbrace{3}_{\text{stage 0}} \longrightarrow \underbrace{4}_{\text{main split}}.$$

The affine linear-part recoveries and distinguished-basis queries use only finite Boolean operations and do not raise these bounds. The linear part of the main episode update is the action of T_e on V . Thus its recovered sequence tests determine T on E^* , since the image of the identity basis vector identifies the monoid value.

Let $A_d = E^* \cap T^{-1}(d)$ be the resulting height-at-most-four tests, and let $F_d = F \cap T^{-1}(d)$ be the height-at-most-one tests from Lemma 7.3. For a language recognized by T with accepting subset $S \subseteq M$, its expression is

$$\bigcup_{\substack{d, f \in M \\ df \in S}} A_d F_f.$$

The factorization lemma and the morphism identity prove that this is exactly the desired language. Concatenation and finite union do not increase the maximum height, which is four. All expressions are over Σ , proving the theorem. $\square$

References

- [1] Françoise Dejean and Marcel-Paul Schützenberger. On a question of Eggan. *Information and Control*, 9(1):23–25, 1966. URL <https://monge.univ-mlv.fr/~berstel/Mps/Travaux/A/1966-6QuestionEgganIC.pdf>.
- [2] Lawrence C. Eggan. Transition graphs and the star-height of regular events. *Michigan Mathematical Journal*, 10(4):385–397, 1963. doi: 10.1307/mmj/1028998975.
- [3] P. Erdős and L. Lovász. Problems and results on 3-chromatic hypergraphs and some related questions. In *Infinite and finite sets, Vol. II*, volume 10 of *Colloquia Mathematica Societatis János Bolyai*, pages 609–627. North-Holland, Amsterdam, 1975. URL https://www.renyi.hu/~p_erdos/1975-34.pdf.
- [4] Wolfgang Krieger. On the subsystems of topological Markov chains. *Ergodic Theory and Dynamical Systems*, 2(2):195–202, 1982. doi: 10.1017/S0143385700001516.
- [5] Tom Meyerovitch. An embedding theorem for multidimensional subshifts. *Ergodic Theory and Dynamical Systems*, 45(6):1870–1914, 2025. doi: 10.1017/etds.2024.117.
- [6] OpenAI. Generalized Star Height at Most Three. OpenAI Math Release preprint [OAI:Generalized-Star-Height-at-Most-Three-September-25-2026](#), 2026.
- [7] OpenAI. Finite Monoid Computations and a Uniform Generalized Star-Height Bound. OpenAI Math Release preprint [OAI:Finite-Monoid-Computations-and-a-Uniform-Generalized-Star-Height-Bound-September-25-2026](#), 2026.

- [8] Jean-Éric Pin, Howard Straubing, and Denis Thérien. Some results on the generalized star-height problem. *Information and Computation*, 101(2):219–250, 1992. doi: 10.1016/0890-5401(92)90063-L. URL <https://www.irif.fr/~jep/PDF/StarHeight.pdf>. Author version dated 12 May 2002.
- [9] Irving S. Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960. doi: 10.1137/0108018.
- [10] Marcel-Paul Schützenberger. On finite monoids having only trivial subgroups. *Information and Control*, 8(2):190–194, 1965. doi: 10.1016/S0019-9958(65)90108-7.
- [11] Howard Straubing. On logical descriptions of regular languages. In Sergio Rajsbaum, editor, *LATIN 2002: Theoretical Informatics*, volume 2286 of *Lecture Notes in Computer Science*, pages 528–538. Springer, Berlin, Heidelberg, 2002. doi: 10.1007/3-540-45995-2_46. URL https://link.springer.com/chapter/10.1007/3-540-45995-2_46.